esprit

Se former autrement

2025 – 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY: INFORMATION TECHNOLOGY

Open Web Catcher:
Design of a Multi-Agent AI Pipeline for Evidence Collection from Illegal Streaming Sites

By: Ahmed ARFAOUI

Academic supervisor: Naadine Maazoun

Corporate Internship Supervisor: Wael Hkiri

Soft Stars

Host organization

uni Tunis
Université de Tunis

cdio

EUR-ACE
Accredited

CTi
Accredited

CONFÉRENCE DES
GRANDES ÉCOLES

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- **Nom & Prénom / Name & Surname :** Ahmed ARFAOUI

Encadrant Entreprise / Business site Supervisor

- **Nom & Prénom / Name & Surname :** Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique / Academic Supervisor

- **Nom & Prénom / Name & Surname :** Naadine Maazoun

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné** / This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Abstract

This report presents the design, development, and evaluation of Open Web Catcher (OWC), an agentic evidence-collection platform for illegal streaming websites. The work was carried out during a PFE internship at Soft Stars in the beIN SPORTS rights-protection context. The objective was to support evidence review by visiting target websites, classifying their pages, extracting hosting and stream evidence, detecting channel signals when visible, and preserving auditable outputs such as screenshots, traces, structured results, and provider metadata.

The internship produced two related artifacts. The first was an exploratory n8n workflow used to validate browser-agent behavior on real dynamic targets and to provide a practical backup path for the existing company crawler. The second was OWC: the complete engineering platform composed of a FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, Puppeteer/Chrome runtime, specialist agents, model telemetry, and review outputs. The internal name Open Web Catcher was used because the platform focuses on catching evidence that is visible only through open-web browser interaction rather than through a static crawler response.

The proposed architecture organizes evidence collection around specialized reasoning agents, browser automation tools, profile-scoped sessions, fingerprint and proxy controls, structured outputs, trace storage, screenshot capture, and evaluation metrics. The database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs; it is not the enterprise production database of the client organization.

Keywords: Open Web Catcher; multi-agent system; browser automation; illegal streaming; evidence collection; stream extraction; channel detection; observability.

Résumé

Ce rapport présente la conception, le développement et l'évaluation d'Open Web Catcher (OWC), une plateforme agentique de collecte de preuves pour les sites de streaming illégal. Le travail a été réalisé dans le cadre d'un projet de fin d'études chez Soft Stars, dans un contexte de protection des droits lié à beIN SPORTS. L'objectif était d'améliorer l'observation des pages suspectes, la classification des pages, l'extraction des liens d'hébergement et des preuves de flux, ainsi que la conservation de traces exploitables pour une revue humaine.

Le projet s'est appuyé sur deux réalisations complémentaires. La première est un workflow n8n utilisé pour valider rapidement l'idée d'un agent capable de naviguer sur des pages dynamiques et de servir de solution de secours au crawler existant de l'entreprise. La seconde est OWC, une plateforme composée d'un backend FastAPI, d'une console Next.js, d'une base PostgreSQL, d'outils MCP basés sur Puppeteer et Chrome, d'agents spécialisés, de métriques d'utilisation des modèles et d'artefacts de revue. L'architecture met l'accent sur la traçabilité, les captures d'écran, les traces d'exécution, la détection de canaux visibles, l'enrichissement fournisseur et l'observabilité.

Mots clés : Open Web Catcher; système multi-agent; automatisation navigateur; streaming illégal; collecte de preuves; extraction de flux; détection de canaux; observabilité.

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor, Naadine Maazoun, for her guidance, feedback, and support throughout this PFE internship. Her remarks helped me keep the report focused on the engineering contribution, the methodology, and the clarity expected from an academic submission.

I would also like to thank my corporate supervisor, Wael Hkiri, for his technical direction, availability, and trust during the project. The discussions around real operational constraints, crawler limitations, browser tooling, deployment choices, and evidence review shaped the work described in this report.

I am grateful to the Soft Stars team for the professional environment, the technical exchanges, and the opportunity to work on a concrete rights-protection problem connected to the beIN SPORTS context. Their feedback helped turn early experiments into a clearer platform design.

Finally, I thank my family and friends for their encouragement and patience during the internship and the preparation of this report.

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASB	Azure Service Bus
CDN	Content Delivery Network
CDP	Chrome DevTools Protocol
DASH	Dynamic Adaptive Streaming over HTTP
DB	Database
DNS	Domain Name System
DOM	Document Object Model
DSRM	Design Science Research Methodology
E2E	End-to-End
HLS	HTTP Live Streaming
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
M3U8	UTF-8 M3U playlist format
MCP	Model Context Protocol
MPEG-DASH	MPEG Dynamic Adaptive Streaming over HTTP
MPD	Media Presentation Description
OWC	Open Web Catcher
OCR	Optical Character Recognition
PFE	Projet de Fin d'Etudes
RDAP	Registration Data Access Protocol
RPD	Requests Per Day
RPM	Requests Per Minute
SSE	Server-Sent Events
SQL	Structured Query Language
TPM	Tokens Per Minute
TTL	Time To Live
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

Acronym	Meaning
UX	User Experience
WHOIS	Domain registration lookup protocol or service

Terminology

Term	Meaning in this report
Open Web Catcher (OWC)	The internal name of the complete evidence-collection platform built during the internship. It refers to the FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, specialist-agent runtime, and review outputs used to catch open-web evidence from browser-visible streaming pages.
Agent	A model-driven runtime component that receives a bounded task, uses approved tools, and returns structured evidence or a failure reason.
Orchestrator	The control component that classifies the starting URL, prepares handoffs, schedules specialist agents, aggregates evidence, and decides the final run status.
Specialist agent	A browser-facing agent with a narrow responsibility, such as classification, landing-page discovery, hosting-page extraction, or embedded-player inspection.
Landing page	A page that lists events, channels, schedules, or match cards and is mainly useful for discovering downstream hosting candidates.
Hosting page	A page that usually owns play buttons, server/source controls, overlays, and redirects before a stream or embedded player becomes visible.
Embedded page	A direct player or iframe page used as a fallback extraction context when the hosting page exposes a player URL.
Browser automation	Programmatic control of a real browser session to inspect rendered pages, click controls, manage frames, capture screenshots, and observe network activity.
Tool call	A browser, provider, memory, or runtime operation requested by an agent and recorded as part of the run trace.
Handoff	A short structured task brief prepared by the orchestrator for the next specialist agent, including target URL, evidence needed, route source, and navigation policy.

Term	Meaning in this report
Trace	The ordered record of runtime events, model calls, tool calls, screenshots, status changes, costs, and final outputs for one investigation.
Observability	The ability to explain what happened during a run after it finishes, including which agent acted, which tools were used, and why a failure occurred.
Stream evidence	A URL, manifest, media request, screenshot, or player state that supports the claim that a target page exposed unauthorized streaming behavior.
Manifest	A playlist or media presentation file, such as HLS or MPEG-DASH, that points a player toward the media segments required for playback.
Iframe	An embedded browser frame that loads a player or third-party page inside the visible hosting page.
Provider enrichment	The step that resolves collected stream hosts or IP addresses into provider, country, network, or abuse-contact information.
Takedown email	A reviewable draft that groups stream evidence, provider details, screenshots, and rights-owner context for later human use.
Regression batch	A repeated set of website tests used to verify that a prompt, model, or tool change fixes new cases without breaking previously working ones.
Evidence artifact	Any persisted output used for review, including screenshots, streams, provider rows, emails, model usage, and tool results.
Runtime event	A timestamped event emitted during execution, such as agent start, tool success, screenshot capture, stream extraction, or failure.
Prompt	The role and task instruction compiled for an agent before it calls the language model.
Memory hint	A reusable site or layout note injected as soft guidance; it can help the agent but must be confirmed against live browser evidence.
Input tokens	Prompt, trace, tool-output, memory, and context text sent to the model during an LLM call.
Cached input tokens	Input tokens reused through the prompt or context cache instead of being billed and processed as fully new context.
Output tokens	Tokens generated by the model as an answer, tool decision, explanation, or structured result.

Term	Meaning in this report
Total tokens	The aggregate token count used by the dashboard to summarize input and output usage for model calls.
Context window	The maximum amount of prompt, trace, tool output, and memory text that the model can consider in one call.
Prompt cache	A cache mechanism used to reuse repeated prompt and context content across similar model calls.
Cache hit rate	The share of model input served from cache within a specific dashboard or model-usage view. It must be read with its denominator.
LLM call	One recorded invocation of the language model, including the prompt, model name, token counters, and cost estimate.
Model spend	The recorded or estimated monetary cost of model calls for a run, batch, or dashboard snapshot.
Cost per run	Model spend normalized by the number of runs in the selected view; it is used to compare evaluation versions.
Tool success rate	Successful observed tool calls divided by observed tool calls in the same telemetry snapshot.
Stream yield	The share of runs that produced at least one stream, manifest, media, or embedded-player evidence candidate.
Strict stream success	A stricter success measure where a run counts only when it reaches reviewable stream evidence, not merely a completed status.
Pipeline run	One end-to-end investigation from a seed URL through classification, specialist routing, evidence collection, enrichment, and final status.
Active run	A pipeline run that is currently queued, running, or waiting on browser/model work.
Queued run	A run or task that has been accepted but has not yet started browser execution.
Run status	The final or current state of a pipeline run, such as success, partial, failed, cancelled, `no_streams`, or `no_hosting_pages`.
`no_streams`	A terminal status used when the browser path was explored but no reviewable media, manifest, stream, or embedded evidence was found.
`no_hosting_pages`	A terminal status used when a landing-page investigation did not discover credible downstream hosting candidates.

Term	Meaning in this report
Evidence bundle	The grouped review output for one run, including streams, screenshots, provider rows, email drafts, status, costs, and trace links.
Provider coverage	The extent to which collected stream or host evidence could be enriched with provider, network, or registration context.
Dashboard snapshot	A live operator-console aggregate at a specific time. It should not be mixed blindly with controlled regression-batch metrics.
Persistence health	A dashboard view showing whether runs, agent runs, runtime events, model calls, tool calls, and outputs are being stored correctly.
Tokenized URL	A media or manifest URL containing temporary query parameters, signatures, or session-bound path values.
Queue message	A structured payload sent through a queue to start work or return results without blocking a synchronous web request.
`agent_tasks`	The Azure Service Bus input queue used in the company deployment flow to receive target URLs and task metadata.
`agent_results`	The Azure Service Bus output queue used in the company deployment flow to return matched URLs and metadata.
Browser profile	A browser context with its own state, cookies, cache, and execution settings for a run or agent.
Profile-scoped session	A browser session tied to one profile so page state can be preserved while avoiding uncontrolled sharing across unrelated runs.
Static crawler	A monitoring component that fetches pages or links without completing the full rendered browser interaction path. In this project it remains useful for discovery but can miss evidence that appears only after scripts, clicks, frames, or playback.
Rendered browser state	The visible and runtime state produced after the browser executes JavaScript, handles frames, loads media, and reacts to user-like actions. OWC treats this state as an evidence surface.

Term	Meaning in this report
Channel detection	The process of identifying a broadcaster or channel signal from landing hints, player labels, screenshots, OCR text, structured agent output, or stream metadata. Landing hints must be verified by hosting or embedded evidence when possible.
Fingerprint profile	A browser identity profile used to keep navigator, viewport, session, and related browser-surface behavior coherent during a run. It reduces obvious automation inconsistencies but does not guarantee bypass of anti-bot systems.
Proxy rotation	The runtime selection of a network proxy for browser execution. In OWC, proxy candidates are validated and kept sticky until failure to avoid breaking session-bound pages.
Sticky proxy	A proxy selected for a browser profile and reused while it remains healthy. This prevents identity changes in the middle of one investigation.

CHAPTER 1

Introduction

This PFE report presents the work carried out at Soft Stars in the context of illegal sports-streaming evidence collection for beIN SPORTS. The project started from a concrete operational need: suspicious pages could be discovered, but the evidence needed for review was often hidden behind browser-rendered interfaces, popups, iframe players, server buttons, redirects, and short-lived media manifests. The internship therefore focused on designing an evidence workflow that could observe the page like a browser user, keep a trace of what happened, and return reviewable evidence instead of only a final URL.

1.1 Host Organization and Internship Context

Soft Stars is a software company working in digital media intelligence, content protection, and technical support for rights-enforcement operations. Its work focuses on helping media organizations observe online distribution channels, identify unauthorized use of protected content, and prepare technical evidence that can be reviewed by operational, legal, or business teams.

The internship took place in a sports media-protection context connected to beIN SPORTS. In this domain, the practical problem is not only detecting that a suspicious website exists. A rights-protection workflow also needs enough evidence to understand what was visible, which channel or event was exposed, which page led to the player, which technical host or provider was involved, and whether a human reviewer can trust the result after the browser session has ended.

1.2 Existing Crawler and Operational Limitations

Before this PFE work, the company already had crawler-based monitoring capabilities. That crawler remained useful for broad discovery, static page collection, and recurring checks over known website families. However, illegal streaming websites increasingly hide the useful evidence behind runtime behavior. A static crawler may see the landing page but miss the hosting page. It may capture the first HTML response but miss the server button that appears after JavaScript execution. It may collect links but miss the iframe, player state, screenshot evidence, channel hint, or final `m3u8` or `mpd` request generated only after a play action.

This limitation explains the move toward an AI-assisted browser workflow. The goal was not to discard crawling. The goal was to add a second path for cases where crawling finds a target but cannot prove the stream path. The n8n landing-agent workflow was therefore deployed by the company as a backup plan beside the crawler: when a target needed browser reasoning, the workflow could open the page, inspect the rendered state, discover match or channel candidates, and return structured URLs and metadata through a queue-based operational flow.

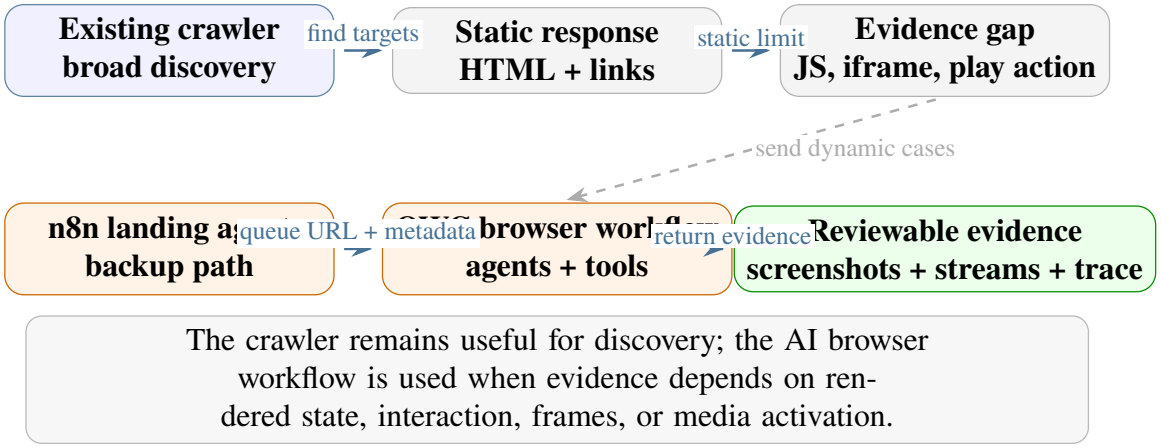


Figure 1.1: How the AI browser workflow complements the existing crawler when static collection misses dynamic evidence.

1.3 Problem Statement

Illegal streaming websites are difficult to investigate because they combine unstable content, dynamic rendering, misleading navigation, popup traps, nested frames, and short-lived media URLs. The evidence path often has several roles:

- a landing page lists matches, events, schedules, or channel cards;
- a hosting page exposes play buttons, server controls, overlays, and iframe candidates;
- an embedded page or direct player exposes the final media behavior;
- a provider-enrichment stage connects collected stream evidence to host, network, country, or abuse-contact information.

If those roles are mixed together, the system becomes hard to debug. A failed result could mean that the page was classified incorrectly, that a landing candidate was missed, that a hosting server was not activated, that an iframe was not inspected, or that the final media request expired. The project therefore required an architecture that separates page roles, specialist agents, browser tools, screenshots, traces, model usage, and final review outputs.

1.4 Internship Mission and Scope

My mission during the internship was to study, design, and validate an evidence collection workflow that complements the existing crawler with browser-based AI reasoning. The work produced two related artifacts.

The first artifact was an n8n workflow. It was used to validate the idea quickly in the company environment and to provide a deployable landing-agent backup path. This workflow helped test whether an agent could inspect rendered pages, follow candidate links, handle page-specific obstacles, and return useful match or hosting metadata.

The second artifact was Open Web Catcher (OWC), the complete platform designed during the internship. The internal name was chosen because the project catches evidence from the open web through browser-visible behavior rather than through a static HTML response. In

this report, OWC refers to the whole project: the FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, Puppeteer/Chrome runtime, specialist agents, model telemetry, and evidence review outputs.

1.4.1 Mission Deliverables

The internship deliverables were:

- a validated n8n landing-agent workflow used as a crawler backup path;
- an OWC architecture separating backend, frontend, database, agents, browser tools, and evidence outputs;
- browser-tool profiles for classification, landing, hosting, and embedded-player investigation;
- support for screenshots, stream candidates, provider enrichment, channel metadata, tool traces, model telemetry, and cost visibility;
- an evaluation and reporting structure based on traceability and reviewability.

1.4.2 Scope Boundaries

The project did not replace the full company enforcement process. It did not automatically send takedown messages, and it did not claim to solve every anti-bot mechanism or every illegal streaming website. The practical scope was to produce an engineering foundation that could investigate dynamic pages, preserve evidence, expose failures clearly, and support future operational integration.

1.5 Main Contributions

The main contributions of the PFE work are:

- a clear analysis of why static crawling is insufficient for many dynamic streaming-evidence paths;
- a DSRM-based artifact design centered on Open Web Catcher;
- an n8n landing-agent workflow deployed as an operational backup beside the existing crawler;
- a multi-agent architecture that separates classification, landing discovery, hosting interaction, embedded-player inspection, provider enrichment, and review output generation;
- a browser-tool layer with profile-scoped inspection, navigation, interaction, screenshot, media extraction, memory, fingerprint, and proxy controls;
- an operator console and persistence model that make traces, screenshots, model usage, tool calls, costs, channel signals, and failure reasons reviewable.

1.6 Report Structure

Chapter 2 presents DSRM as the sole methodology and derives the project requirements. Chapter 3 introduces the technical background needed to understand streaming pages, browser automation, media manifests, agent workflows, and evidence traceability. Chapter 4 describes the OWC architecture, including the agent responsibilities, browser tooling, fingerprint and proxy handling, channel detection, persistence, and reviewability. Chapter 5 explains the implementation path from the n8n backup workflow to the OWC platform and the company deployment flow. Chapter 6 presents evaluation and testing results. Chapter 7 provides the general conclusion.

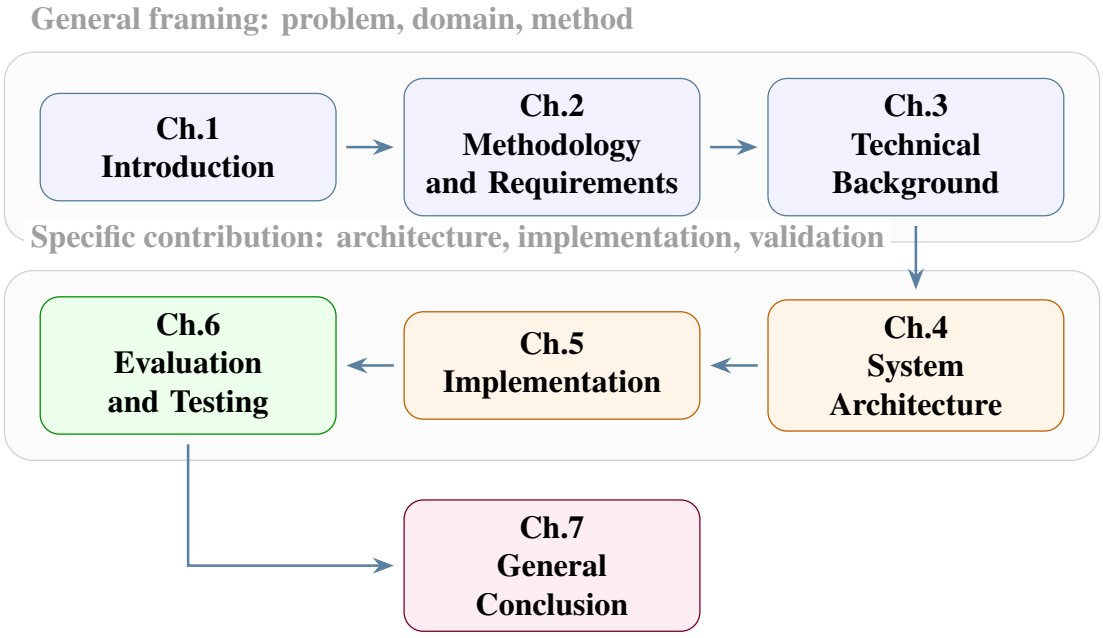


Figure 1.2: Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.

CHAPTER 2

Methodology, Requirements Analysis, and Design Constraints

The internship was organized around Design Science Research Methodology (DSRM). This choice is deliberate: the work did not consist of studying a phenomenon from the outside or training a predictive model. It consisted of designing, building, demonstrating, and evaluating an engineering artifact for a concrete organizational problem. In this report, that artifact is Open Web Catcher (OWC), the evidence collection platform built around browser tooling, specialist agents, trace storage, screenshots, provider enrichment, and reviewable outputs.

DSRM provides the backbone for the chapter structure that follows. The problem is first identified and motivated, then solution objectives are defined, the artifact is designed and developed, the artifact is demonstrated, the artifact is evaluated, and finally the work is communicated through this PFE report and its diagrams. This phase logic follows the design-science view of information-systems artifacts proposed by Hevner et al. [19] and the DSRM process described by Peffers et al. [20].

2.1 DSRM as the Methodological Backbone

DSRM was selected because it gives equal importance to the organizational problem and to the engineered artifact. A static crawler already existed in the company context, but it was limited when the useful evidence appeared only after browser rendering, interaction, iframe traversal, server switching, or media activation. The PFE work therefore needed a methodology that could justify a new artifact, not only describe a work plan.

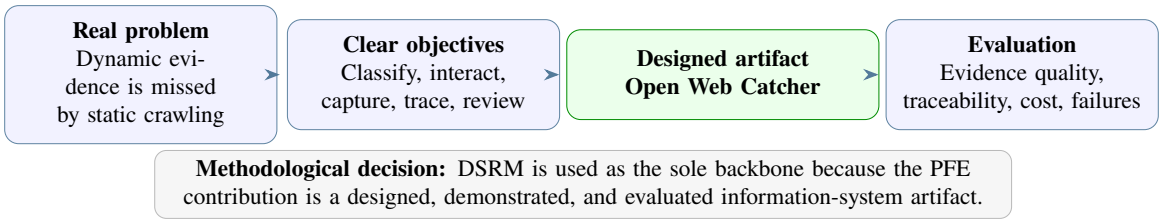


Figure 2.1: DSRM selection logic for an artifact-centered PFE project.

2.1.1 Why DSRM Fits the Internship

The project has three properties that make DSRM suitable. First, it is artifact-centered: the final contribution is a platform and workflow design, not a pure analysis. Second, the artifact is evaluated through use: it is meaningful only if it can inspect real pages, produce traces, capture screenshots, and return evidence. Third, the organizational relevance is explicit: Soft Stars

needed a more adaptable way to complement crawler-based monitoring when illegal streaming pages became too dynamic for static extraction.

2.1.2 DSRM Phases Used in the Report

The six DSRM phases are not treated as abstract theory. Each one corresponds to work carried out during the internship. Figure 2.2 summarizes the adopted backbone.

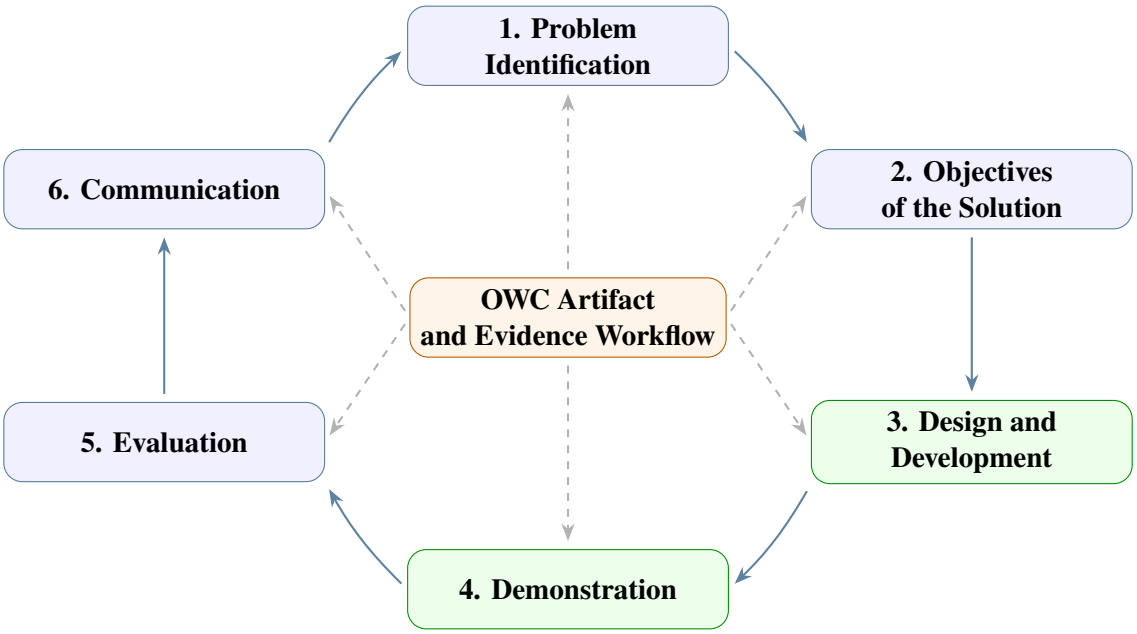


Figure 2.2: DSRM backbone used to structure the OWC problem, artifact, demonstration, evaluation, and communication.

2.1.2.1 Problem identification and motivation

The first phase established the practical problem: suspicious streaming pages could be found, but collecting reviewable evidence was difficult. Some pages exposed only lists of matches or channels. Others required a click on a server button before a player loaded. Others hid the player in an iframe or generated short-lived manifest URLs only after playback. A crawler that reads static HTML can miss these states because it does not perform the same sequence of visual and network actions as a browser user.

2.1.2.2 Objectives of the solution

The second phase translated the problem into objectives. The artifact needed to ingest a URL, classify the page role, follow landing-to-hosting and hosting-to-embedded handoffs, activate players when necessary, collect screenshots, detect stream evidence, preserve channel and provider metadata when available, and store enough trace data for later review. These objectives became the functional and non-functional requirements listed later in this chapter.

2.1.2.3 Design and development

The third phase produced the artifacts. The n8n workflow was used first because it allowed fast operational validation of an AI-assisted landing workflow and could act as a backup path beside the existing crawler. OWC then formalized the same ideas in a maintainable platform with FastAPI, Next.js, PostgreSQL, MCP browser tooling, Puppeteer/Chrome, profile-scoped sessions, model telemetry, and specialist agents.

2.1.2.4 Demonstration

The fourth phase demonstrated that the artifact could operate on representative page types: landing pages, hosting pages, embedded player pages, popup-heavy pages, tokenized manifest cases, and pages where channel evidence had to be verified from visible player or screenshot context. Demonstration was not limited to a happy path; it included runs where the system returned explicit failures such as no hosting pages, no streams, or skipped downstream enrichment.

2.1.2.5 Evaluation

The fifth phase evaluated the artifact through trace inspection, tool telemetry, screenshots, token and cost records, stream yield, provider enrichment, and dashboard snapshots. The aim was not to claim that every illegal streaming page can be solved. The aim was to verify whether the artifact made the extraction process more observable, repeatable, and reviewable than a one-shot crawler or a manually inspected page.

2.1.2.6 Communication

The final phase is the communication of the artifact. This report presents the problem, methodology, technical background, architecture, implementation, evaluation, and final limitations. The diagrams and tables are included to make the artifact understandable as an engineering contribution, not to reproduce internal source files.

2.2 DSRM Mapping to the Internship Work

Table 2.1 maps each DSRM phase to the work performed during the internship and to the concrete outputs used in the rest of the report.

2.3 Project Evolution Timeline

The timeline follows the DSRM logic but remains concrete. The internship started with evidence needs and crawler limitations, moved to browser-agent experiments, then to the n8n landing-agent backup workflow, and finally to the OWC platform architecture and evaluation.

2.4 Functional Requirements

DSRM phase	Application in this project	Main output
Problem identification and motivation	Analyze crawler limitations, dynamic page behavior, stream activation, iframe traversal, channel ambiguity, and the need for reviewable evidence.	Problem statement and motivation for OWC
Objectives of the solution	Define the extraction, traceability, screenshot, channel, provider, cost, and review requirements that the artifact must satisfy.	Functional and non-functional requirements
Design and development	Build the n8n validation workflow, then design OWC as a platform with backend, frontend, database, browser tools, agents, and observability.	n8n workflow and OWC architecture
Demonstration	Run the artifact on representative landing, hosting, embedded, popup, iframe, and tokenized-stream scenarios.	Demonstrated evidence paths and failure paths
Evaluation	Inspect traces, screenshots, stream evidence, provider enrichment, tool behavior, token usage, costs, and dashboard telemetry.	Evaluation metrics, results, and limitations
Communication	Produce the PFE report, diagrams, final PDF, and design explanation for academic and company review.	Written report and final synthesis

Table 2.1: DSRM phase mapping to project activities and outputs.

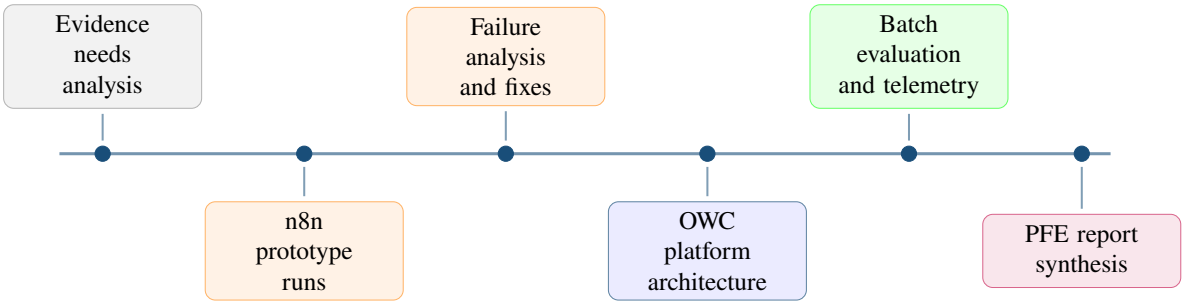


Figure 2.3: Internship timeline from evidence needs analysis to evaluated OWC architecture.

The functional requirements describe what the artifact must do to support the evidence-collection workflow.

ID	Requirement summary	Priority
FR-1	URL ingestion and run lifecycle management	**Critical**
FR-2	Page-type classification using rendered browser evidence	**Critical**
FR-3	Orchestrator handoff from classification to landing, hosting, or embedded agents	**Critical**
FR-4	Landing-page discovery of match, event, channel, and hosting candidates	**High**
FR-5	Hosting-page interaction with server/source controls, overlays, and play buttons	**Critical**
FR-6	Embedded-player inspection through frame trees, player state, and media signals	**Critical**
FR-7	Screenshot capture at important evidence and failure points	**High**
FR-8	Stream evidence extraction for manifests, media URLs, and embedded fallbacks	**Critical**
FR-9	Channel detection from landing hints, player context, screenshot/OCR text, and structured agent output	**High**
FR-10	Provider enrichment and reviewable evidence bundle generation	**Medium**
FR-11	Operator console views for runs, telemetry, screenshots, tools, settings, and provider results	**High**

Table 2.2: Functional requirements derived from the evidence-collection workflow.

2.5 Non-Functional Requirements

The non-functional requirements explain the qualities required for the artifact to be useful after a run finishes.

This DSRM framing defines the rest of the report. Chapter 3 introduces the technical concepts needed to understand the artifact. Chapter 4 describes the OWC architecture, including the agent routing, browser tool layer, fingerprint and proxy controls, and channel-detection path. Chapter 5 then explains how those decisions were implemented.

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	OWC separates backend routes, frontend views, agent contracts, browser tools, runtime settings, and persistence records.
NFR-2	Observability	Runs persist runtime events, agent runs, LLM calls, tool calls, screenshots, streams, provider rows, and final statuses.
NFR-3	Traceability	Each result can be connected back to a URL, agent, tool call, screenshot, stream candidate, and final decision.
NFR-4	Reliability	Profile-scoped sessions, bounded retries, popup recovery, iframe recovery, media verification, and explicit failure states reduce silent errors.
NFR-5	Cost control	Token counters, cached input tracking, model settings, rate-limit awareness, and dashboard cost summaries make model usage visible.
NFR-6	Adaptability	Page-type specialists, browser profile controls, proxy configuration, fingerprint rotation, and memory hints support changing site behavior.
NFR-7	Deployment fit	The company deployment keeps n8n as a backup operational orchestrator and runs the landing agent through queue-driven container jobs.

Table 2.3: Non-functional requirements and implementation responses.

CHAPTER 3

Technical Background

This chapter introduces the technical concepts needed before presenting the system architecture and implementation. It deliberately stays at a general level: the goal is to explain the web structures, streaming mechanisms, browser automation tools, and agentic concepts that make the project understandable. The concrete architecture, database schema, prompts, and tool contracts are described later.

The chapter moves from the target websites themselves toward the technologies used to analyze them. It starts with page structures and DOM representation, then explains streaming protocols and embedded players, then covers browser automation issues such as ads, cookies, popups, and anti-bot providers. It ends with the agentic concepts used in the project: agents, prompts, tool use, workflow automation, LangChain, MCP servers, model providers, caching, and token usage.

3.1 Streaming Website Structures

Illegal streaming websites are not a single page type. They are usually a chain of pages with different responsibilities: one page lists events, another page presents a selected match and server choices, and a final embedded page or iframe hosts the player. Understanding these page roles is necessary before discussing extraction tools, because each role exposes different evidence and requires different interaction.

3.1.1 The normal page flow

Most target websites can be described as a three-step navigation path. A landing page contains match listings. A hosting page focuses on one selected match and usually offers one or more streaming servers. An embedded page contains the actual player, often inside an iframe loaded from another domain.

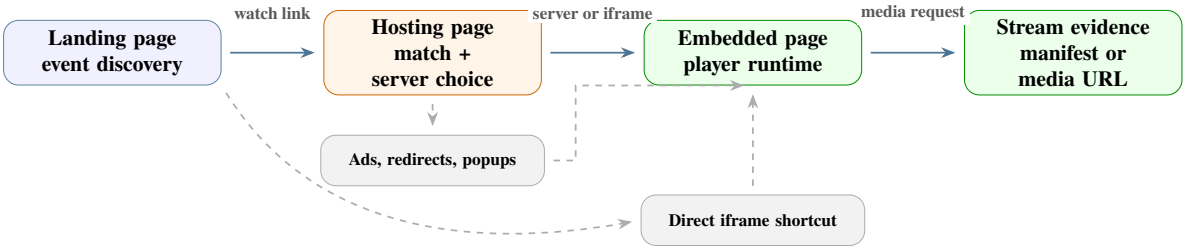


Figure 3.1: Common navigation flow from event listing to hosting page and embedded player.

This flow is not always clean. Some sites skip the hosting page and link directly to an embedded player. Others place several ad pages between the hosting page and the player. Still,

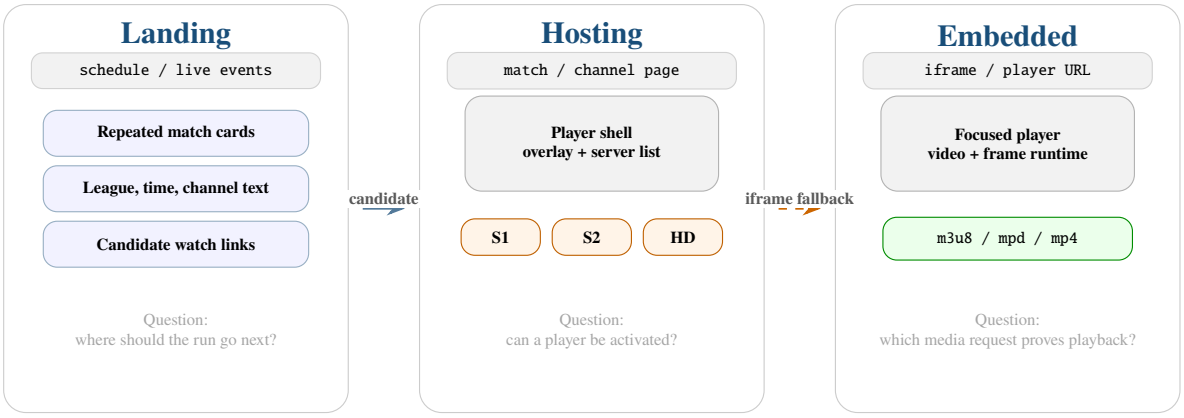


Figure 3.2: Comparison of the three page roles used by the extraction workflow.

this model is useful because it separates discovery, server selection, and media extraction into distinct technical problems.

3.1.2 Landing pages

Landing pages are the discovery layer. They usually contain match cards, schedule rows, league filters, date tabs, search boxes, or category menus. The useful output from a landing page is not the stream itself but a set of candidate hosting links and match metadata.

Common landing-page layouts include:

- **card grids**: repeated blocks with teams, time, league, and a watch link;
- **tables or lists**: one row per event, often with a direct action link;
- **tabbed schedules**: live, today, yesterday, tomorrow, or league-specific tabs loaded through JavaScript;
- **lazy-loaded sections**: match cards that appear only after scrolling or after an XHR request;
- **embedded JSON**: page data stored in `<script type="application/json">` blocks or `data-*` attributes.

The important idea is that landing pages are structurally repetitive. Even when class names change, the page still tends to contain repeated event units. This makes layout recognition more useful than relying on one fixed CSS selector.

3.1.3 Hosting pages

Hosting pages are the decision layer. They normally focus on one match and contain a player container, a server list, tabs such as `Server 1` or `HD`, and sometimes channel names or broadcaster logos. A hosting page may already expose the stream URL, but more often it only exposes an iframe or a JavaScript player configuration.

Typical hosting-page signals are:

- one match title or two team names repeated near the player;
- server buttons, mirrors, or quality selectors;

- an iframe whose `src` points to a player page;
- player libraries such as Video.js, JWPlayer, Clappr, hls.js, or dash.js;
- visible state changes after clicking play, selecting a server, or closing an overlay.

Hosting pages are more difficult than landing pages because they require interaction. The same click can activate a real player, open a popup, switch a server, or trigger a redirect. This is why browser automation and runtime observation become necessary.

3.1.4 Embedded pages

Embedded pages are the media layer. They are often minimal pages loaded inside iframes, with little content except a player, a play button, and scripts that request the final media manifest. They may belong to another domain, which means the top page and the player page have different DOMs, cookies, and network contexts.

An embedded page is important because the final stream reference is often generated only there. A static request to the hosting page may not show any `.m3u8` or `.mpd` URL, while the embedded player requests the manifest after user activation.

Page type	Main role	Useful evidence
`landing_page`	Discover events and candidate match pages	match links, team names, schedule metadata, listing screenshot
`hosting_page`	Select server and activate player	server labels, iframe URLs, player state, page screenshot
`embedded_page`	Load the final media runtime	stream manifest, segment requests, player diagnostics
`other`	Filter noise such as ads, redirects, and unrelated pages	reason for skip or blocked state

Table 3.1: Streaming-site page taxonomy and evidence role.

The page taxonomy turns a messy website into a sequence of smaller questions: Where are the matches? Which page hosts the selected match? Which server or iframe contains the player? Which runtime request proves that media was loaded?

3.2 DOM Structures and Web Page Representation

The Document Object Model (DOM) is the browser's tree representation of a web page. It turns HTML into nodes such as documents, elements, attributes, text nodes, iframes, buttons, images, and scripts. Browser automation tools operate on this tree when they query selectors, read attributes, click buttons, or remove overlays.

3.2.1 A simplified DOM tree

The diagram below shows a simplified DOM structure. It is intentionally basic, but it captures the idea used throughout the project: a visible page is not just text, it is a tree of nested elements with attributes and event handlers.

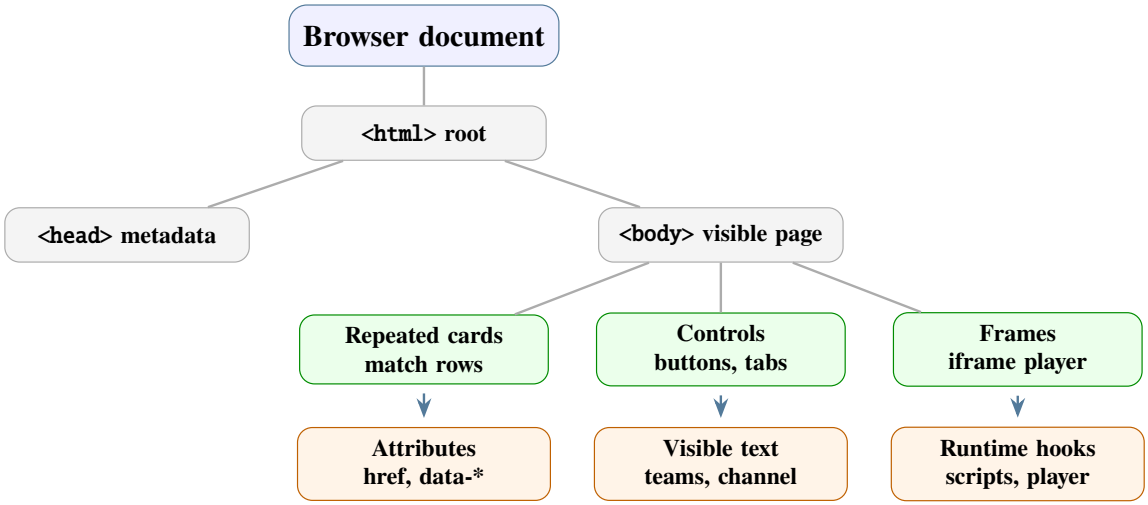


Figure 3.3: Simplified DOM tree showing how a browser represents an HTML document.

For streaming sites, the DOM is useful but incomplete. It can reveal match cards, buttons, iframes, scripts, and player containers, but it may not reveal media URLs that are generated at runtime. This is why DOM inspection is combined with screenshots and network monitoring.

3.2.2 DOM signals on streaming pages

Different page roles expose different DOM signals. Landing pages expose repeated containers. Hosting pages expose player containers and server controls. Embedded pages often expose a reduced tree with one `<video>` element or a JavaScript player root.

DOM signal	Where it appears	Why it matters
Repeated cards or rows	landing pages	indicates match listings and candidate links
`<a href="...">`	landing and hosting pages	provides navigation targets
`data-*` attributes	landing and dynamic pages	may store event IDs, team names, or hidden URLs
`<iframe src="...">`	hosting pages	points to embedded player context
`<video>` or `<source>`	hosting or embedded pages	may expose direct media source references
script text and JSON blocks	all page types	may contain player configuration or lazy-loaded data

Table 3.2: DOM signals used to understand streaming-site pages.

The conclusion is that DOM analysis is a starting point, not a complete solution. It identifies page structure and interaction targets, while runtime browser evidence confirms what actually happened after the page executed.

3.3 Streaming Protocols and Embedded Players

The final evidence in this domain is usually not a normal web page. It is a media manifest, a player configuration, or a network request made by the browser while the player is active. This section explains the main protocols and player embedding patterns at a general level.

3.3.1 Common embedding patterns

Hosting pages usually embed players in one of three ways. The simplest case is a direct `<video>` element with a `src` or nested `<source>` tag. A common case is an iframe that loads a player from another domain. A more dynamic case is a JavaScript player library that inserts the video source after page load.

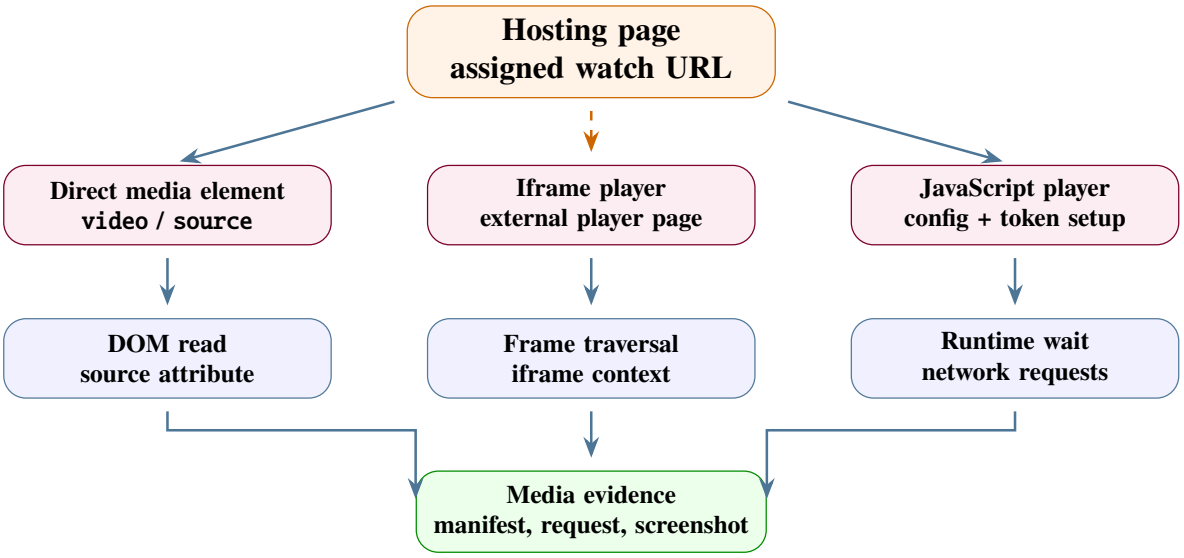


Figure 3.4: Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.

This distinction matters because each embedding type requires a different observation strategy. Direct video tags can be read from the DOM. Iframes require frame traversal. JavaScript players often require runtime execution and network monitoring.

3.3.2 HLS and MPEG-DASH

HTTP Live Streaming (HLS) is one of the most common protocols for web video streaming. Its entry point is a `.m3u8` playlist that points either to variant playlists or to media segments. MPEG-DASH uses an XML `.mpd` manifest that describes periods, representations, and segment addressing rules. Both protocols are adaptive: the player can choose different quality levels depending on network conditions [33, 34].

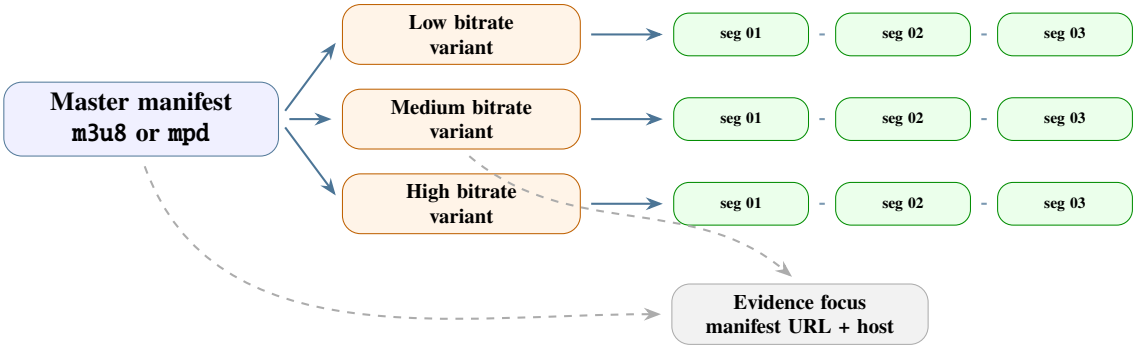


Figure 3.5: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

For evidence collection, the manifest URL is usually more important than downloading media segments. It proves that the page resolved to a playable stream entry point and often contains CDN, token, and path information.

3.3.3 Tokens, cookies, referers, and provider checks

Streaming manifests are frequently protected by short-lived tokens, cookies, referer checks, or provider-side gates. A URL such as `playlist.m3u8?token=...` may work only for a few minutes and only from the browser session that generated it. CDNs and fronting providers such as Cloudflare can also enforce challenge pages, bot checks, cache rules, and origin protection [32].

This is why the browser session matters. A raw HTTP request may fail even when the browser can play the stream, because the browser carries the active cookies, headers, referer, and JavaScript-generated state. The technical background therefore includes both media protocols and web-provider behavior.

3.4 Browser Automation, Scraping, and Runtime Obstacles

Static scraping means downloading HTML and parsing it. That is useful for simple pages, but it is not enough for pages that depend on JavaScript, user clicks, iframes, or runtime media requests. Browser automation tools such as Puppeteer and Playwright execute pages in real browsers and expose APIs for clicking, reading the DOM, taking screenshots, and listening to network activity [12, 13, 18].

3.4.1 Static crawler versus rendered browser state

A crawler is efficient when the target evidence is present in the fetched HTML or in simple linked pages. Its limitation appears when the evidence is created only after a runtime sequence: opening a collapsed channel group, clicking a server tab, activating the player, waiting for an iframe, and then observing the network. In those cases, the browser state is the evidence surface, not only the page source. This distinction is the technical reason why OWC adds browser agents beside the existing crawler instead of treating the crawler as sufficient.

3.4.1.1 Rendered state as evidence

Rendered state includes visible text, player overlays, frame trees, screenshots, media elements, and network requests that appear after interaction. It matters because the final stream or channel signal may not exist at the first page load. A screenshot, media-state record, or tool trace can therefore explain why an agent believed the page exposed unauthorized streaming behavior.

3.4.2 Puppeteer as a browser control layer

Puppeteer provides programmatic control over Chromium. In this project context, the important concept is not only "click a button" but full runtime observation:

- inspect the DOM after JavaScript has executed;
- click buttons, tabs, overlays, server selectors, and play controls;
- traverse frames and iframes;
- capture screenshots for human review;
- observe network requests through the Chrome DevTools Protocol;
- read browser-side state with `page.evaluate(...)`.

Puppeteer is therefore the bridge between a high-level extraction objective and the actual state of the web page. It lets the system test what a user would see and what the browser actually requested.

3.4.3 Ads, cookies, popups, and other annoyances

Illegal streaming websites often monetize through aggressive ad behavior. These behaviors are not just inconvenient; they change the extraction problem because the automation may click the wrong layer, follow an ad redirect, or lose the original player context.

Obstacle	Common symptom	General handling idea
Popups and new tabs	clicking play opens an ad page instead of the player	close extra targets and retry on the original page
Overlay layers	a fake play button covers the real player	detect visible overlays and click or remove them carefully
Cookie banners	controls are blocked until consent is handled	accept, dismiss, or record as blocking evidence
Ad iframes	DOM contains many unrelated frames	rank frames by player-like signals instead of following every iframe
Cloudflare-like challenges	browser sees an interstitial instead of the page	wait, retry with normal pacing, or preserve a blocked-state result

Table 3.3: Browser-level obstacles and handling strategies.

Ad blocking tools such as uBlock Origin can reduce noise by blocking known ad and tracking domains [35]. However, they cannot replace browser reasoning. If a site serves ads and player

assets from the same domain, aggressive blocking can break the player itself. The general solution is layered: block obvious noise, keep popup recovery, and preserve evidence when the page remains blocked.

3.4.4 Why screenshots and network logs matter

DOM text alone does not prove that a player actually loaded. A screenshot can show that the player was visible or that a challenge blocked the page. A network log can show that the browser requested a media manifest. Together, DOM, screenshot, and network evidence give a more reliable picture than any one source by itself.

This is also why extraction tools should avoid claiming success only because a keyword or button was found. In this domain, useful evidence is the combination of page role, visible state, player behavior, and media request.

3.5 Agentic Evidence Collection Concepts

The project uses the term *agentic* to describe an extraction approach where a language-model-driven agent observes a page, chooses a tool action, checks the result, and repeats until it has enough evidence or reaches a limit. This differs from a fixed scraper because the next action depends on the current browser state.

3.5.1 Agents and ReAct-style loops

Agentic systems commonly use a loop that combines reasoning and tool use. In a ReAct-style pattern, the agent observes the state, reasons about what to do, acts through a tool, and verifies the result before continuing [1, 3].

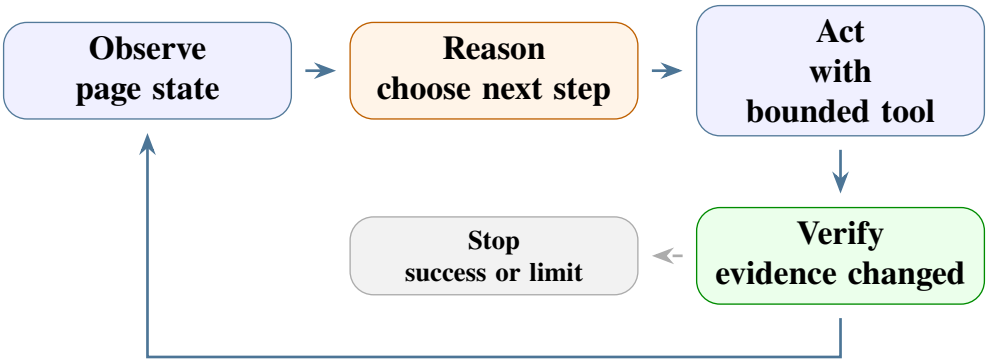


Figure 3.6: Observe–act–verify loop used by browser agents.

For streaming sites, this loop is useful because the agent can adapt when a server button fails, a popup opens, an iframe appears, or a page turns out to be unrelated. The agent is not expected to know the full path in advance; it uses tools to discover it.

3.5.2 Prompt engineering, tokens, and token usage

Prompt engineering is the process of defining the agent's role, constraints, tool usage rules, evidence requirements, and output format. In this project domain, a prompt must prevent premature conclusions. It should tell the agent to ground decisions in visible page state, DOM evidence, screenshots, and network observations.

Tokens are the units used by language models to process prompts and outputs. Long page snapshots, repeated tool results, and verbose traces increase token usage and therefore increase cost and latency. Token management is part of the technical background because browser agents can easily spend too much context on noisy HTML, ads, or repeated page states.

Practical token-control ideas include:

- summarize page observations instead of passing full raw HTML every time;
- keep only evidence-relevant DOM snippets and network URLs;
- stop early when the page role or stream evidence is clear;
- separate input tokens, output tokens, and cached tokens when estimating cost;
- use provider-side caching when stable prompt sections are reused.

LLM providers such as Gemini expose model features that affect cost, context length, streaming behavior, pricing, and caching [6, 10, 11]. Provider caching is especially relevant when the same system instructions, tool descriptions, or long context blocks are reused across many runs.

3.5.3 Workflow automation tools

Workflow automation tools let developers connect nodes for HTTP requests, browser actions, LLM calls, data transformations, and external services through a visual flow. They are useful for fast validation because a workflow can be assembled and adjusted before a full application is built. In this report, workflow automation is treated as an orchestration concept: it sequences steps, passes data between nodes, calls model providers, and makes evidence-collection logic visible during experimentation. The specific workflow tool used during implementation is introduced in Chapter 5.

3.5.4 LangChain, tools, and MCP servers

LangChain is a framework for building applications around language models, prompts, tools, memory, and structured chains. At a general level, it helps connect an LLM to external capabilities: a browser tool, a database lookup, a parser, a memory store, or a custom function. This is useful when the model should not only answer textually but take controlled actions [23].

The Model Context Protocol (MCP) is a way to expose external tools and resources to an agent through a consistent interface. In this type of project, an MCP server can expose browser tools such as inspect, click, navigate, screenshot, harvest, or media-state checks. The agent does not need to know how each tool is implemented internally; it needs a clear tool contract, input schema, and output shape [25].

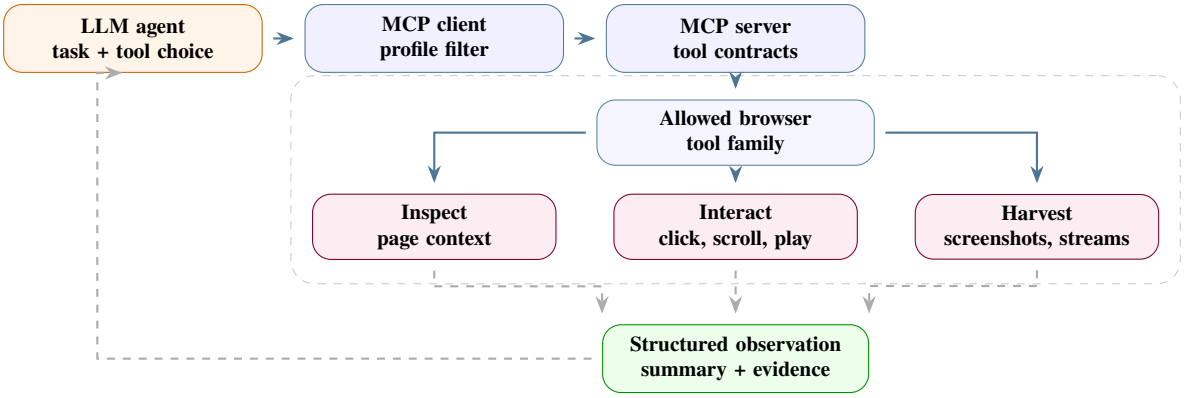


Figure 3.7: General relationship between an LLM agent, profile-scoped MCP tools, and structured browser observations.

The important concept is separation of responsibility. The LLM decides what to try next, while tools perform bounded actions and return structured evidence. This keeps the agentic process auditable.

3.6 Chapter Summary

This chapter established the technical background for the rest of the report. It explained the page flow from landing pages to hosting pages and embedded players, the DOM structures used to inspect those pages, the streaming protocols and player embeds that carry the final evidence, and the browser-level obstacles caused by ads, cookies, popups, iframes, and provider-side checks. It also introduced the agentic concepts used later: ReAct-style loops, prompt engineering, tokens, provider caching, workflow automation, LangChain, and MCP tool servers.

The next chapters use these concepts more concretely. Chapter 4 translates them into architecture, and Chapter 5 explains how the implementation applies them in the actual evidence-collection system.

CHAPTER 4

System Architecture

This chapter presents the architecture of Open Web Catcher (OWC), the evidence-collection platform developed during the internship. It starts with the user-facing system and runtime components, then narrows into the multi-agent pipeline, the handoff contract between agents, the responsibilities of each agent, and the browser-tool layer that lets agents read and act on rendered pages. It closes with the data that makes a run reviewable. Chapter 5 then maps these architectural decisions to the implementation.

Chapter focus

The system is a staged evidence pipeline, not a single unrestricted browser agent. FastAPI starts and records runs. The orchestrator owns the LangGraph route, memory lookup, queues, handoffs, aggregation, provider analysis, and email draft generation. Four browser-facing specialists perform bounded work: classification, landing-page discovery, hosting-page extraction, and embedded-player extraction. Each specialist receives a narrow task, uses only its profile-scoped MCP browser tools, returns typed output, and leaves events that explain both success and failure. Channel detection is handled by the agents as part of evidence interpretation, while the browser runtime carries the practical controls required by this domain: rendered-page context retrieval, screenshot capture, profile-scoped sessions, fingerprint handling, and proxy rotation.

4.1 System Purpose and Architecture Reading Path

The platform is an evidence-collection workbench for illegal streaming websites. The operator launches investigations and reviews the resulting evidence. The engineer or administrator maintains models, prompts, pricing, datasets, browser runtime settings, and MCP tool configuration. External services provide target website content, model inference, screenshot storage, and provider ownership data.

The architecture should be read in four layers:

1. **Product layer**: the operator console launches runs, monitors progress, and reviews evidence.
2. **Runtime layer**: FastAPI validates runtime readiness, schedules jobs, resolves settings, and persists trace data.
3. **Agent layer**: the orchestrator routes work through specialized agents instead of asking one prompt to do every task.
4. **Evidence layer**: streams, screenshots, provider rows, emails, events, model calls, and tool calls are normalized for later review.

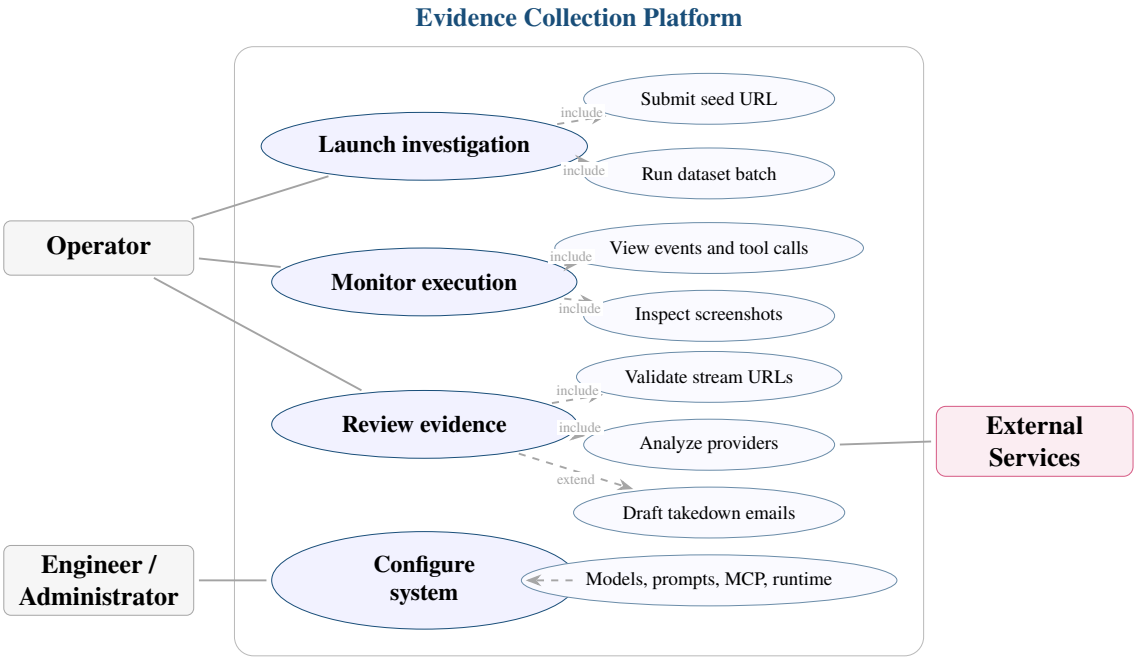


Figure 4.1: Use-case overview of the evidence-collection platform.

Use-case group	User intent	System response
Launch investigation	Submit one URL, a single-agent test, or a dataset batch.	Create a traceable run, validate runtime readiness, and schedule execution.
Monitor execution	Follow agent transitions, tool calls, screenshots, events, and failures.	Stream active state and persist each important runtime event.
Review evidence	Inspect streams, screenshots, provider analysis, metrics, and takedown drafts.	Assemble a reviewable evidence bundle with final status and supporting trace data.
Configure system	Tune models, prompts, MCP tools, browser settings, pricing, and datasets.	Persist configuration and expose health or preflight feedback before runs start.

Table 4.1: Use-case responsibilities across operator and administration workflows.

4.2 Global Runtime Architecture

The runtime separates the operator console, API, agent execution, browser automation, persistence, and external enrichment services. The Next.js console does not access the database directly; it calls FastAPI endpoints. FastAPI owns background-job execution, active trace assembly, settings resolution, and database access. Browser work is delegated to profile-scoped MCP tooling so the agents receive only the tools allowed for their page type.

This separation is practical rather than cosmetic. Browser automation, LLM reasoning, dashboard rendering, and persistence fail in different ways. Keeping their boundaries visible

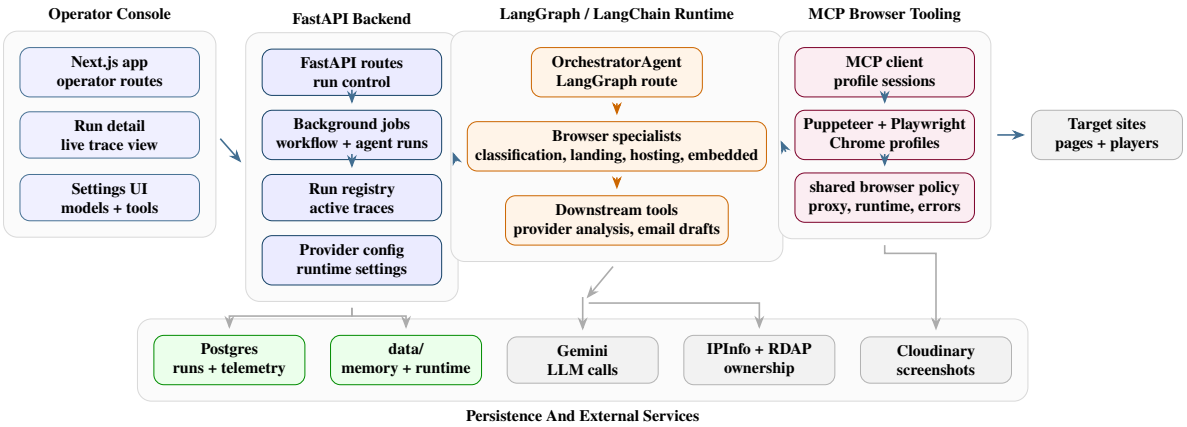


Figure 4.2: Component architecture linking the operator console, backend, agent runtime, MCP tooling, persistence, and external services.

Runtime area	Main responsibility	Why it is separate
Operator console	Launch workflow runs, inspect run detail, manage datasets, providers, tools, browser settings, and model configuration.	The UI stays thin; it consumes API payloads instead of owning persistence.
FastAPI backend	Validate preflight state, create jobs, run background work, assemble active and persisted traces, and expose review payloads.	Runtime control and database access remain testable from backend routes.
Agent runtime	Execute the orchestrator graph and the specialist model/tool loops.	Routing policy remains explicit instead of hidden inside one prompt.
MCP browser tooling	Provide profile-scoped page, frame, screenshot, interaction, and network tools.	Browser dependencies and page failures are isolated from the API process.
Persistence and services	Store normalized run records and call Gemini, Cloudinary, IPInfo, RDAP, and local runtime files.	The final answer and the failure path can be reviewed after execution.

Table 4.2: Global runtime boundaries used by OWC.

makes failures easier to classify: a browser-tool timeout, a model retry, a missing stream, a skipped provider lookup, and a dashboard payload issue are different problems.

4.3 End-to-End Agent Pipeline

At large scale, the platform behaves as a controlled multi-agent pipeline, not as one unrestricted browser agent. The operator gives the system a seed URL. The API starts a run, creates an active trace, and calls the orchestrator. The orchestrator checks memory as soft guidance, calls classification, routes to the appropriate specialist path, collects evidence, and only then runs provider analysis and email generation.

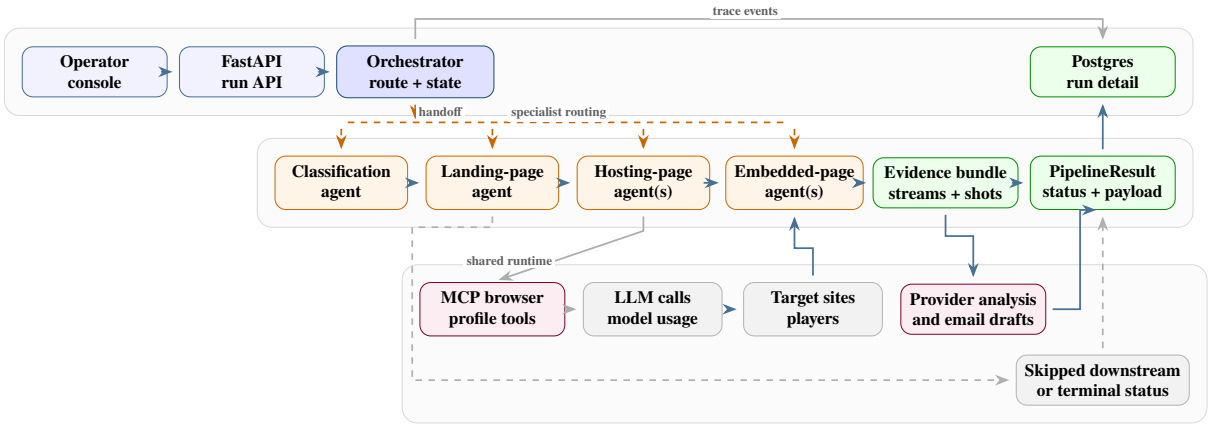


Figure 4.3: End-to-end agent architecture at workflow scale.

The sequence view shows the same workflow as a runtime contract. A run starts as an API request, becomes a background job and active trace, executes through the orchestrator graph, records model and tool activity, and is finally returned to the console as normalized run-detail data.

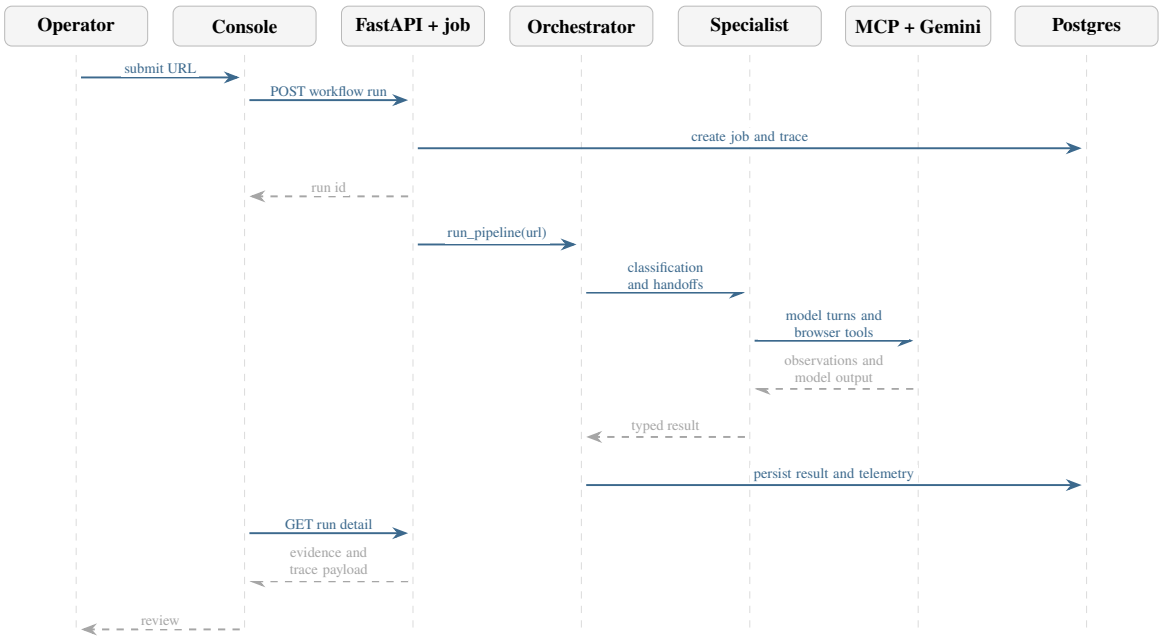


Figure 4.4: Sequence diagram for launching, executing, persisting, and reviewing a workflow run.

The pipeline has three important rules:

- **Agents do not call each other directly**: every specialist returns typed output to the orchestrator.
- **The orchestrator owns queues**: landing can add hosting or embedded targets, hosting can add more hosting or embedded targets, and embedded returns final media evidence.
- **Downstream stages require streams**: provider analysis and email generation are skipped when no stream URL exists.

4.4 Agent Communication and Orchestrator Handoffs

The orchestrator is the routing authority. It builds the LangGraph state machine, checks memory as soft guidance, calls classification, prepares handoffs, executes landing, hosting, and embedded specialists as needed, aggregates structured outputs, then runs provider analysis and email generation only when stream evidence exists. This keeps failures interpretable: a classification timeout, missing hosting candidate, hosting-player failure, embedded-player failure, and provider skip are different operational states.

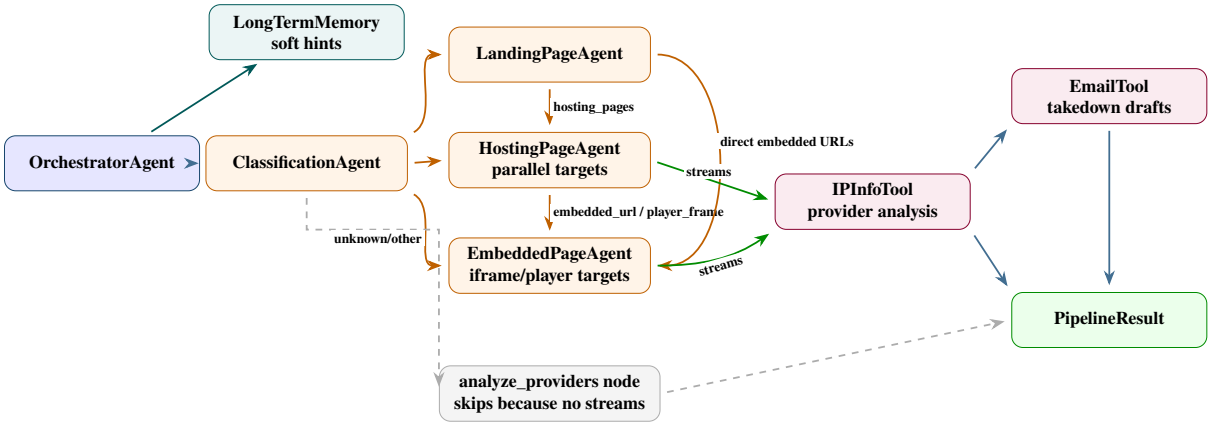


Figure 4.5: Agent interconnection graph showing orchestrator-owned routing and specialist responsibilities.

The handoff is the main communication contract between agents. It contains the root URL, target URL, page type, classification reasoning, route source, memory hints, navigation policy, required evidence, and known pattern context. A specialist receives this context inside its task brief, performs only its bounded job, and returns a schema that the orchestrator can normalize and route.

In this design, communication is visible at three levels. First, route decisions are graph edges, so the next stage is explicit. Second, handoff text gives each specialist the local context it needs without making it own the whole pipeline. Third, the observer records agent starts, model calls, tool calls, screenshots, status changes, costs, and final rollups so the dashboard can explain what happened.

Prompt compilation is also part of the communication contract. Each browser-facing specialist receives the same architectural ingredients but a different stage contract. The compiled prompt combines the base policy, the agent-specific contract, the URL task brief, runtime context, long-memory hints, and short-memory working state. Short memory is run-local: it tracks recent navigation, tool calls, selectors, candidates, streams, server records, screenshots, and the next working objective. Long memory is cross-run site memory: it stores summarized playbooks in the site-memory database and profile JSON so a future run can receive hints without trusting them blindly. Both are soft context; live browser evidence still decides the route.

Stage	Main input	Responsibility	Main output
Orchestrator	Seed URL, settings, observer, and memory hints.	Build the LangGraph route, prepare handoffs, aggregate results, compute final status, and call downstream tools.	`PipelineResult`, trace events, provider rows, and email drafts.
Classification	URL and profile-scoped browser context tools.	Decide page type without extracting streams or provider data.	Classification result with page type, confidence, and reasoning.
Landing	Landing URL plus orchestrator handoff.	Discover watch or hosting candidates while preserving match, iframe, channel, and player hints.	`ExtractionResult` with match records and hosting or embedded candidates.
Hosting	Assigned hosting URL plus evidence checklist and navigation policy.	Operate server/source controls, activate playback, collect network evidence, and return explicit embedded handoffs when needed.	Screenshots, server rows, stream URLs, diagnostics, or embedded player URLs.
Embedded	Verified player or iframe URL plus handoff context.	Stay inside the assigned player context, recover from drift, and harvest final stream evidence.	Stream URLs, screenshots, frame diagnostics, and player state.
Provider and email tools	Stream URLs, provider rows, screenshots, and extractions.	Enrich stream hosts and draft reviewable takedown notices.	`ProviderInfo` records and `TakedownEmail` drafts.

Table 4.3: Agent and tool responsibility map.

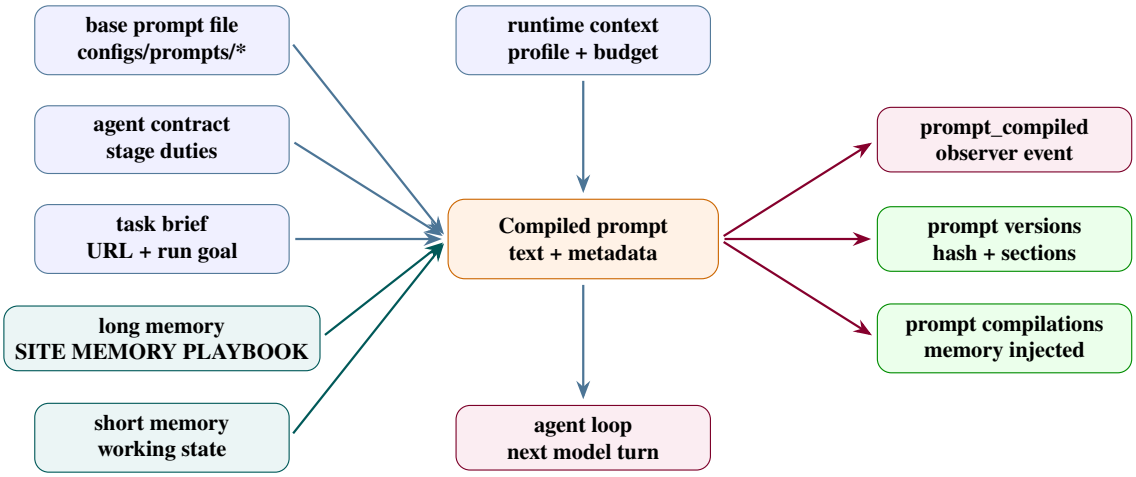


Figure 4.6: Prompt compilation and memory inputs for each browser-facing specialist.

4.5 Agent-by-Agent Design

4.5.1 Orchestrator Agent

The orchestrator is the control plane for one run. It owns state, queues, handoffs, aggregation, provider and email execution, final status, and trace emission, but it does not inspect pages directly.

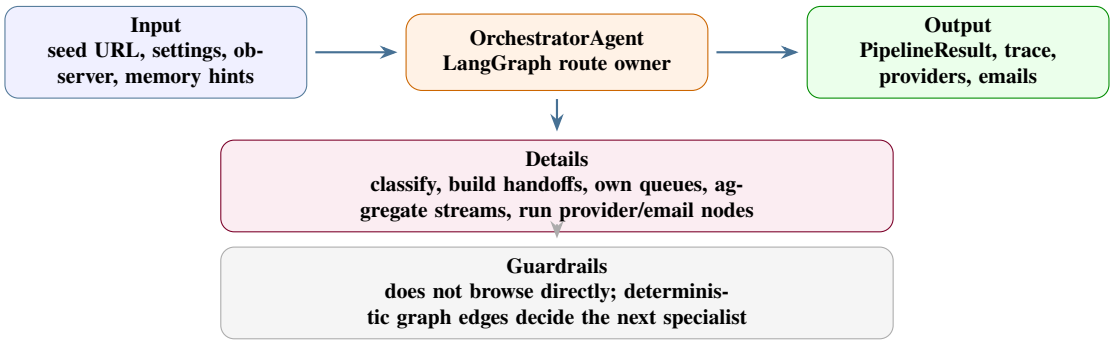


Figure 4.7: Orchestrator-agent design card.

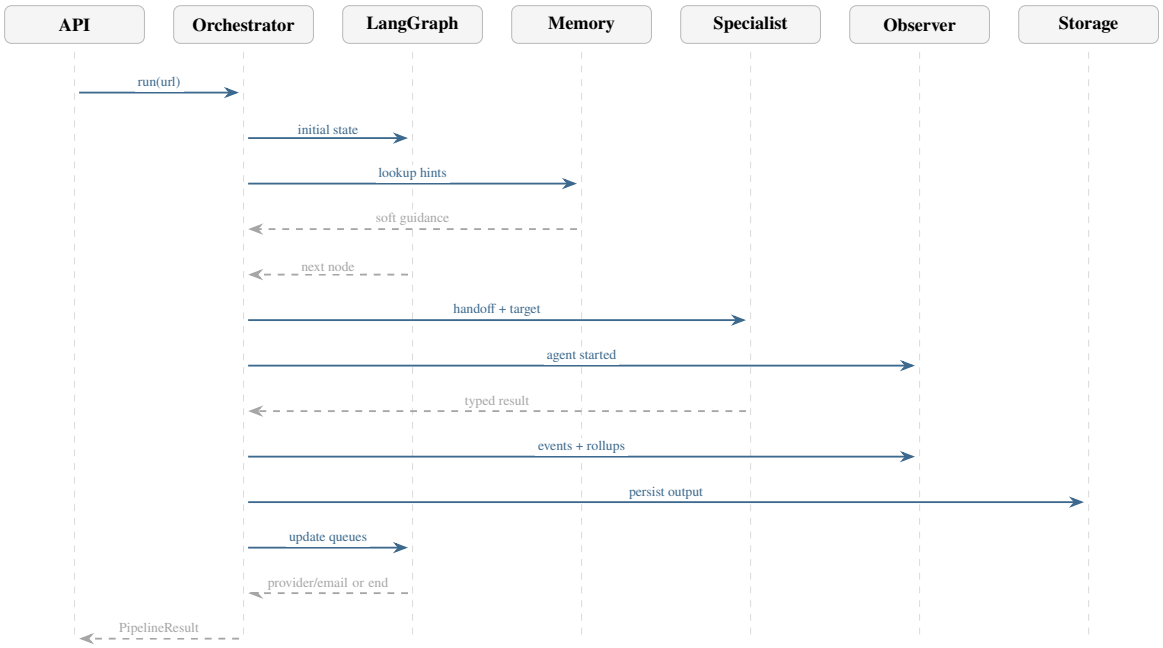


Figure 4.8: Sequence diagram for orchestrator handoff, trace recording, and route updates.

4.5.2 Classification Agent

The classification agent answers one question: what type of page is this URL? It distinguishes landing, hosting, embedded, unknown, and irrelevant pages before extraction begins.

A landing result sends the run to the landing agent. A hosting result queues the root URL for the hosting agent. An embedded result queues the root URL for the embedded agent only when

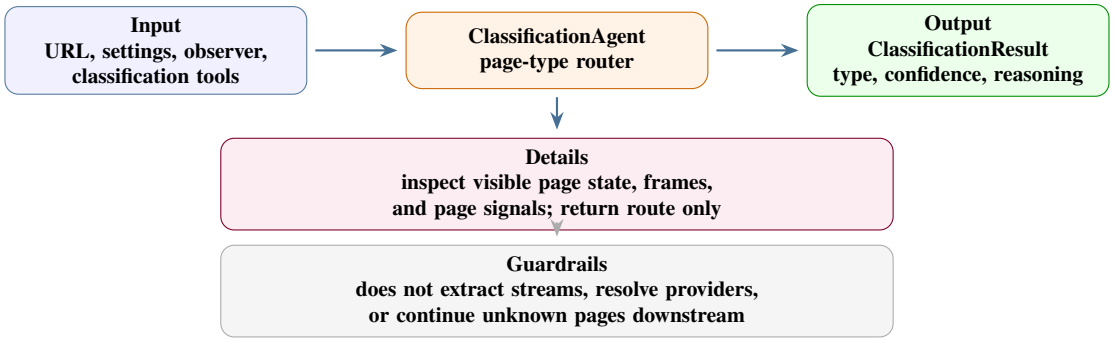


Figure 4.9: Classification-agent design card.

the page looks like a direct player. Unknown or irrelevant pages skip provider and email stages because no stream evidence exists.

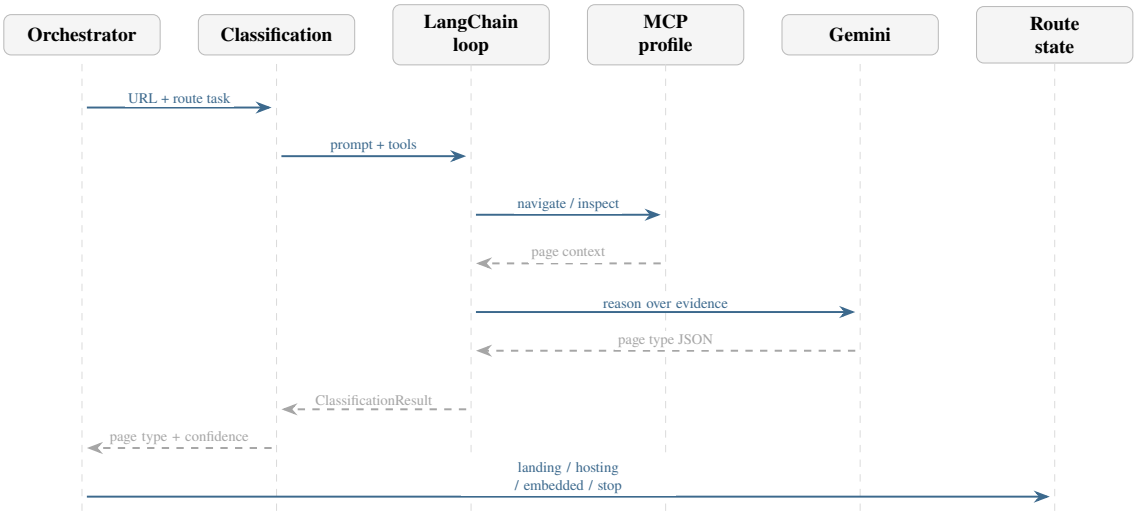


Figure 4.10: Classification-agent sequence: inspect page state and return a routing decision.

4.5.3 Landing-Page Agent

The landing-page agent turns broad site pages into bounded downstream targets. It discovers useful watch, hosting, embedded, or direct-stream candidates from homepages, schedule pages, directory pages, and match listings.

The orchestrator deduplicates landing outputs. Normal watch or server pages go to the hosting queue, direct player URLs go to the embedded queue, and provider-like media URLs are preserved as stream evidence.

4.5.4 Hosting-Page Agent

The hosting-page agent owns watch pages and server/source selection pages. It activates players, operates server controls, handles overlays, captures screenshots, and returns streams or explicit embedded-player handoffs.

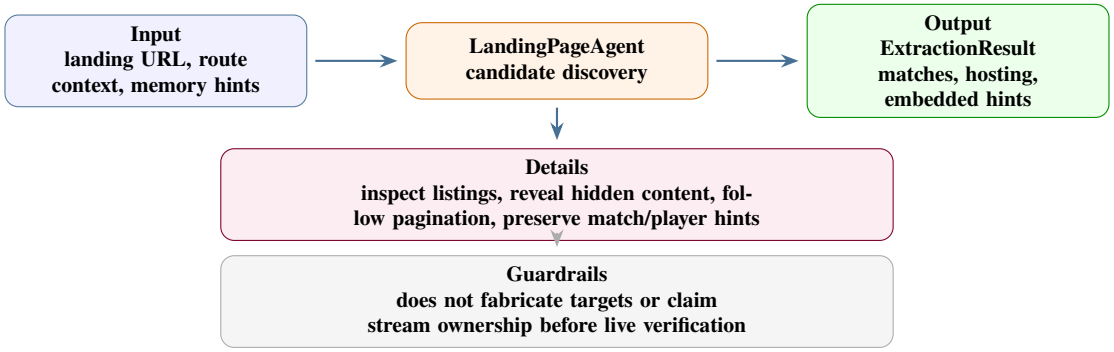


Figure 4.11: Landing-page-agent design card.

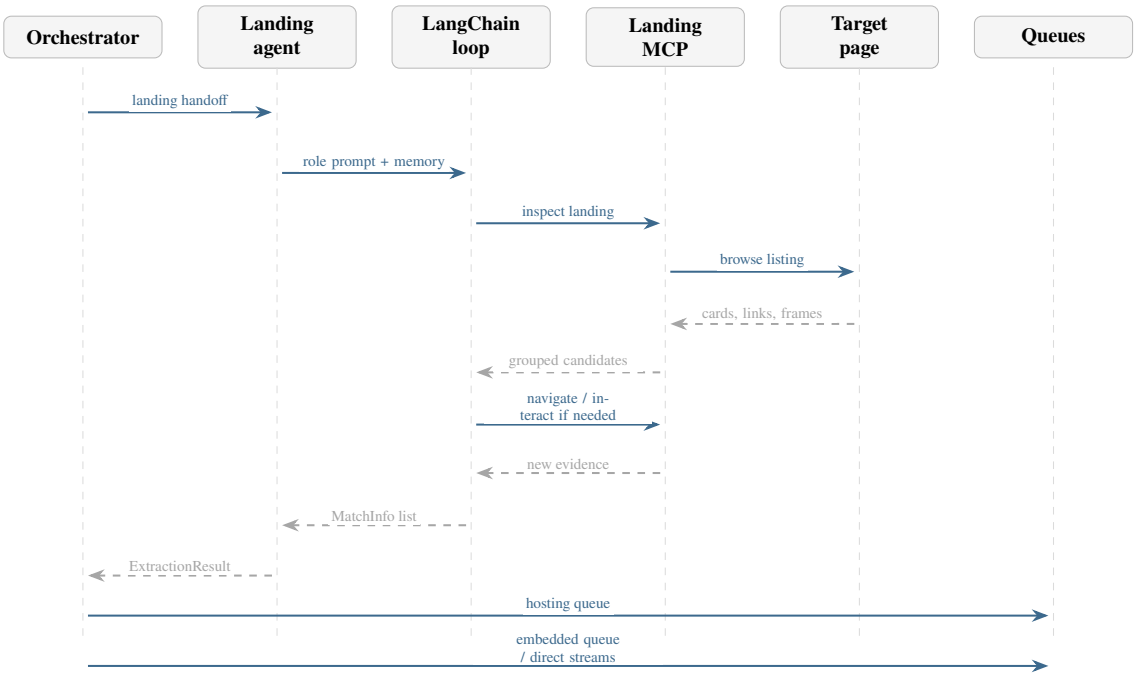


Figure 4.12: Landing-agent sequence: convert listing-page evidence into downstream queues.

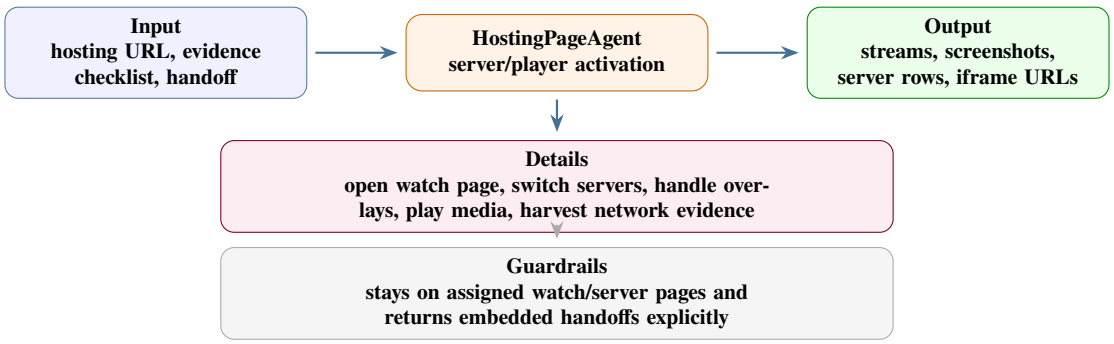


Figure 4.13: Hosting-page-agent design card.

If the page exposes an iframe or direct player URL instead of a stream, the hosting result carries that URL as an embedded handoff. If playback fails, it still returns diagnostics so the run explains why extraction stopped.

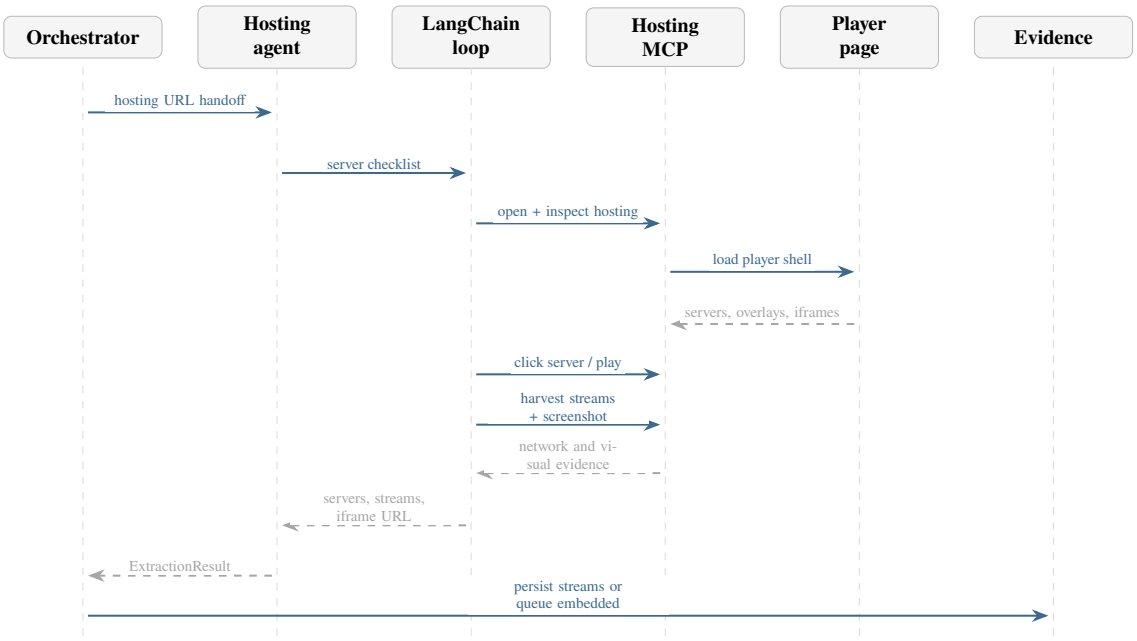


Figure 4.14: Hosting-agent sequence: activate watch pages and return stream or iframe evidence.

4.5.5 Embedded-Page Agent

The embedded-page agent owns direct player or iframe URLs. It stays inside the assigned player context, inspects frames, activates iframe-local controls, recovers from popup or redirect drift, and collects final media evidence.

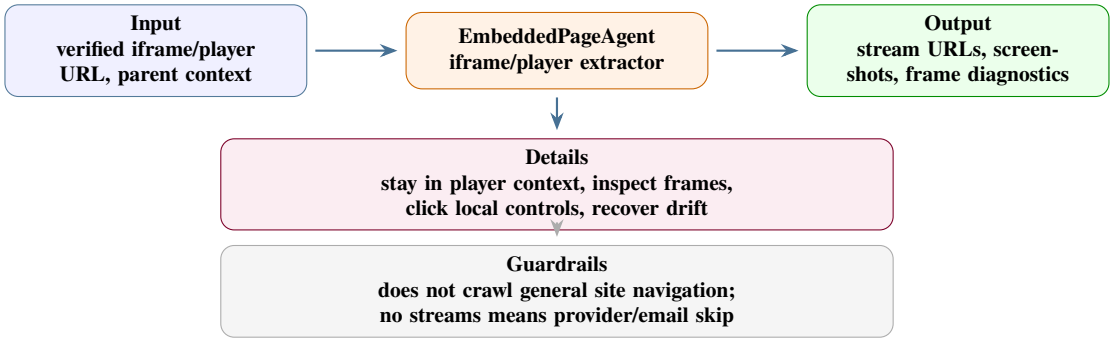


Figure 4.15: Embedded-page-agent design card.

4.5.6 Channel Detection Across Agents

Channel detection is an agent responsibility, not a standalone browser tool. The tools expose page text, DOM structure, frame state, screenshots, server labels, and media requests; the agents decide whether those signals are strong enough to become evidence. This distinction matters

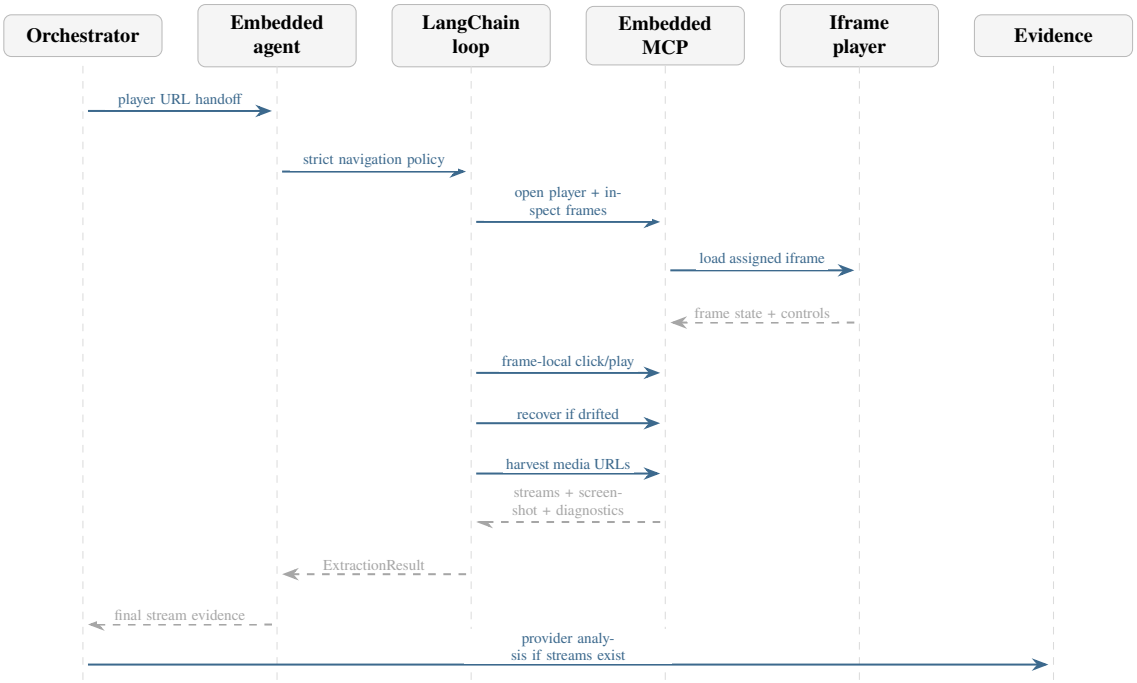


Figure 4.16: Embedded-agent sequence: stay in the player context and harvest final media evidence.

because a channel name printed on a landing page is only a hint, while a channel label near an activated player or inside the embedded player context is stronger evidence.

The responsibility is shared across the route. Classification can preserve obvious channel text from the first page observation. The landing agent keeps channel and match hints attached to candidate hosting URLs. The hosting agent verifies whether the selected server or player still corresponds to the same channel. The embedded agent has the final opportunity to confirm, refine, or reject the channel signal from iframe-local text, player overlays, and screenshot evidence. The orchestrator then normalizes these observations as candidates, final channel fields, and confidence notes in the pipeline result.

Agent stage	Channel evidence it can read	How the signal is treated
Classification	Page title, visible headings, URL text, and broad page context.	Preserved as an initial hint, not accepted as final proof.
Landing	Match cards, league labels, event rows, revealed tabs, and candidate hosting links.	Attached to each downstream candidate so the route keeps its sports context.
Hosting	Server labels, player container text, active source state, screenshots, and network-adjacent player metadata.	Used to verify or override landing hints after the watch page is activated.
Embedded	Iframe-local DOM, player overlays, frame title, screenshot text, and final media evidence context.	Treated as the strongest page-level confirmation before provider analysis.

Table 4.4: Channel detection responsibilities across the specialist agents.

4.5.7 Provider Analysis and Email Generation

Provider analysis and email generation are downstream evidence-packaging stages. They are not page-navigation specialists. Provider analysis reads the collected stream URLs, resolves host or IP ownership, and returns provider information and abuse contact data. Email generation groups provider rows and stream evidence into reviewable takedown drafts. These drafts are not sent automatically; they are artifacts for human review.

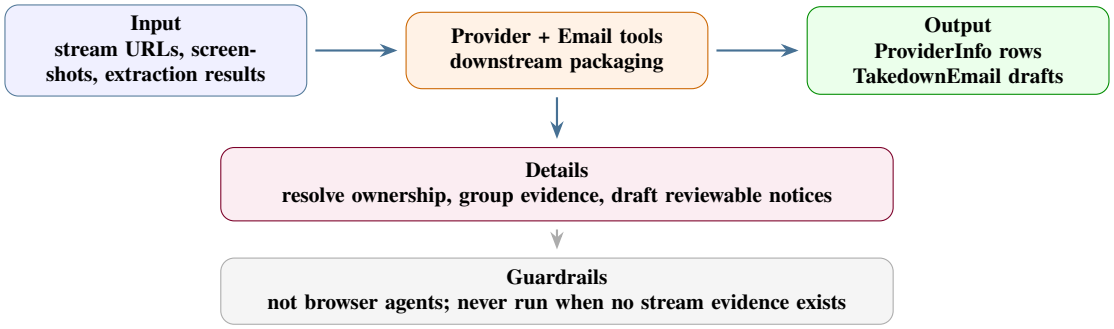


Figure 4.17: Provider-analysis and email-generation design card.

4.6 Browser Tooling and Runtime Controls

After the agent responsibilities are clear, the browser-tool layer explains how the agents observe and change real web pages. OWC does not ask an agent to reason from raw HTML alone. Illegal streaming pages are usually rendered by JavaScript, split across nested frames, and guarded by overlays, popups, server tabs, tokenized URLs, or delayed media requests. For that reason, every browser-facing specialist receives a profile-scoped MCP tool surface backed by Puppeteer and Chrome.

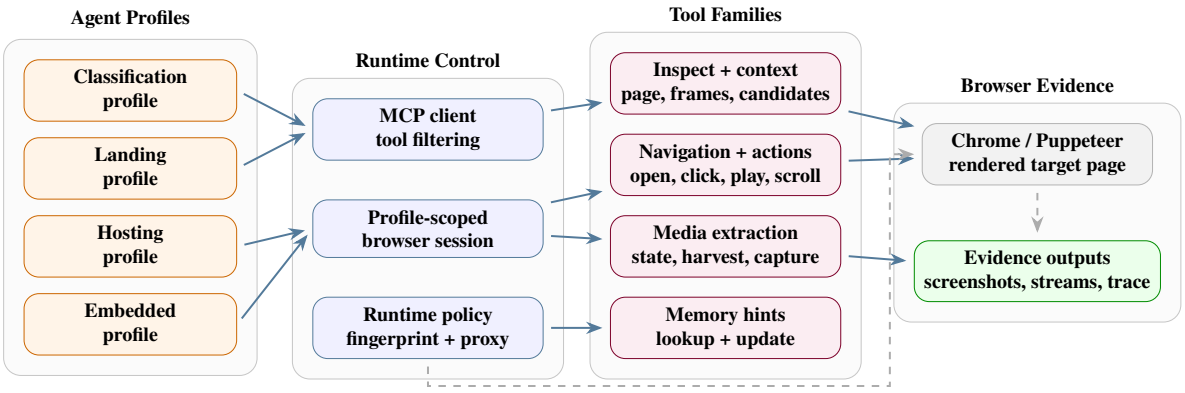


Figure 4.18: Profile-scoped MCP browser tooling and runtime controls used by OWC agents.

4.6.1 Tool Families

The Puppeteer tool layer is divided into tool families so that each agent can ask for the smallest useful observation or action. This keeps context growth under control: the model

receives a compact view of the rendered page, then requests a narrower element, frame, media, or screenshot operation when it needs proof.

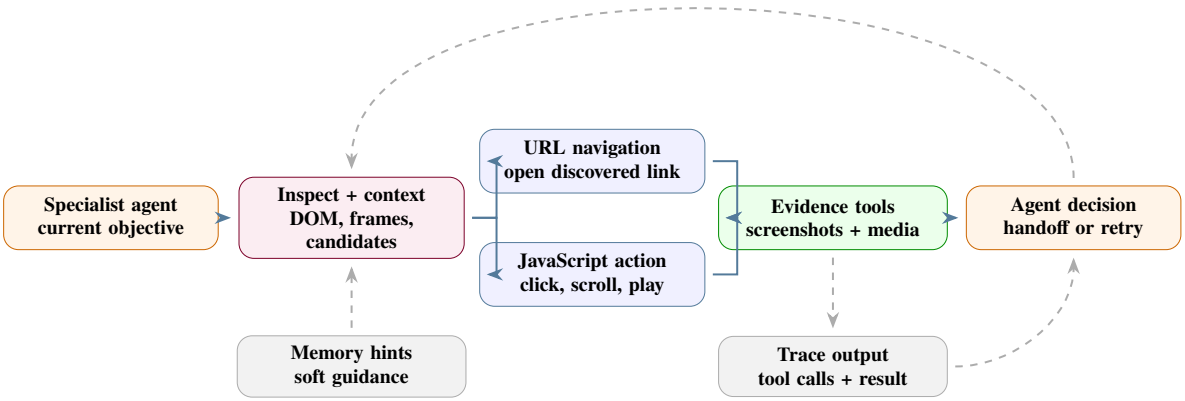


Figure 4.19: Global browser-tool flow: inspect first, choose URL navigation or JavaScript action, then validate evidence.

4.6.1.1 Inspection and context retrieval

Inspection tools are the first read layer. They inspect the rendered Document Object Model (DOM), not only the first server response. Broad inspectors summarize visible text, grouped anchors, buttons, candidate cards, server controls, iframe trees, player containers, media clues, and screenshot links. Page-type inspectors specialize that same idea: the landing inspector looks for match cards and hosting candidates, the hosting inspector looks for server/source controls and player state, and the embedded inspector focuses on iframe-local player evidence.

Context tools are narrower. When an agent sees a possible target in a broad observation, it can request element details, a selector-specific subtree, a frame tree, candidate element references, or media state. This avoids repeatedly sending a large DOM summary back to the model. The architectural rule is simple: broad context is used to choose where to look, and scoped context is used before acting.

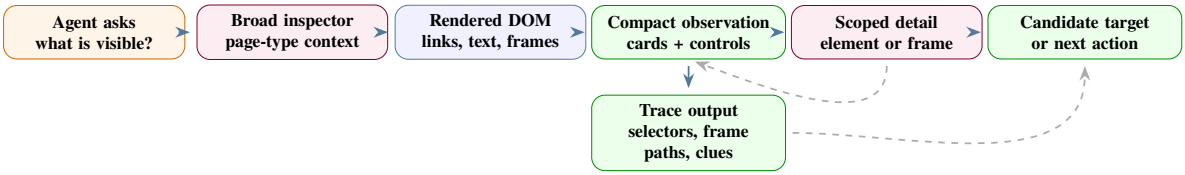


Figure 4.20: Inspection and context-retrieval flow from rendered DOM observation to scoped targets.

4.6.1.2 Navigation and interaction

Navigation has two different meanings in OWC. When inspection returns a real link or candidate URL, the agent uses URL navigation: open the link, wait for the document to settle, then inspect the new page. When the page requires JavaScript to reveal content, the agent stays in the same browser context and uses interaction tools: click a tab or server button, scroll to

trigger lazy loading, type into an input, select a control, or attempt playback. Both paths must be followed by a new observation because either a new document or a changed DOM state can invalidate the previous context.

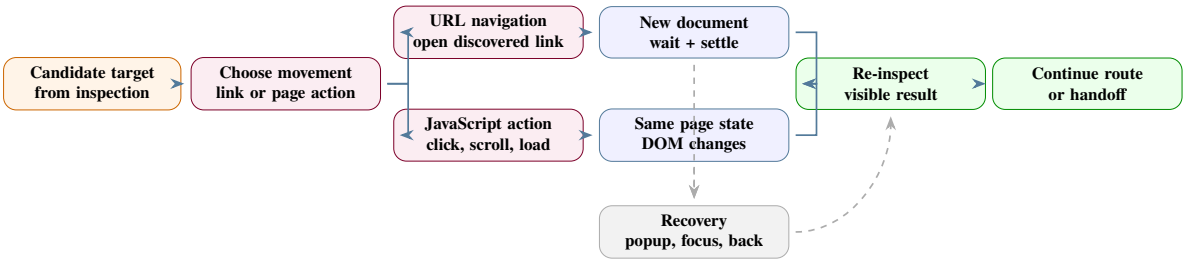


Figure 4.21: Navigation flow split between direct URL navigation and JavaScript-driven page interaction.

4.6.1.3 Evidence and media extraction

Evidence tools convert browser state into reviewable artifacts. Screenshots show what the page looked like after navigation or interaction. Media-state tools confirm whether a player is present, whether playback started, and whether the active frame changed. Stream-harvesting tools collect `m3u8`, `mpd`, `mp4`, or related candidates from network and browser signals. The agent still decides whether the candidate is acceptable evidence, but the tools provide the structured facts that make the decision auditable.

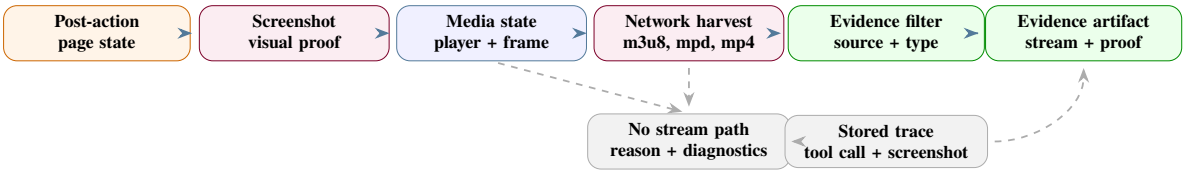


Figure 4.22: Evidence and media-extraction flow from page state to reviewable artifacts.

4.6.1.4 Memory and site hints

Memory tools read and update reusable site hints such as known server tabs, useful selectors, common iframe patterns, and previous failure causes. They do not replace live browser evidence. A memory hint can tell an agent where to start, but the agent must still inspect the current page, act through Puppeteer if needed, and validate the result with screenshots, player state, or stream evidence.

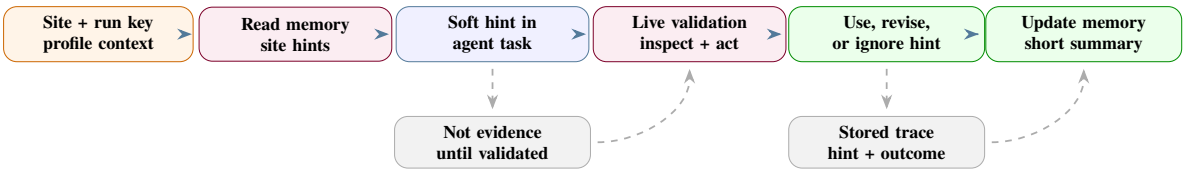


Figure 4.23: Memory-tool flow showing hints as guidance that must be validated live.

4.6.2 Fingerprint Handling and Proxy Rotation

OWC treats browser identity as a runtime setting rather than as prompt text. Browser profiles keep cookies, cache, and session state scoped to the current run or agent profile. Fingerprint handling controls properties such as browser identity, navigator surface, viewport-related behavior, and profile fallback so that repeated browser sessions do not expose an obviously inconsistent automation profile.

Proxy rotation is handled at runtime as a validated sticky strategy. Candidate proxies can be collected from configured remote or custom sources, validated against a test endpoint, and kept for the current browser profile while they remain healthy. Rotation occurs after failure instead of on every click, because changing network identity during one investigation can break cookies, session-bound player URLs, and tokenized media links. If no proxy candidate works, the runtime can fall back to direct browsing instead of incorrectly reporting the target site as unreachable.

Runtime concern	Architectural rule	Reviewable output
Tool scope	Each specialist receives only the tools needed for its page role.	Tool calls and profile identifiers attached to the run trace.
Browser profile	State is scoped by browser/profile so cookies, cache, and session-bound pages stay coherent during a run.	Profile, runtime settings, and browser events.
Fingerprint handling	Browser identity is hardened and rotated according to runtime policy, not improvised by the agent.	Runtime configuration and browser diagnostics.
Proxy rotation	Proxies are validated and kept sticky until failure; direct fallback avoids false site-down conclusions.	Proxy strategy, failures, and final navigation status.
Context retrieval	Agents request broad DOM summaries first, then scoped element, frame, or media details before acting.	Inspector output, selectors, frame paths, and observation events.

Table 4.5: Browser runtime controls that support reliable and reviewable evidence collection.

4.7 Data, Trace, and Reviewability View

The reviewability view is stored as normalized database records, not only as one large final JSON object. The central class is `PipelineRunRecord`. It stores the stable run identity, root URL, final status, aggregate counts, token totals, cost totals, and high-level classification fields. Background jobs can exist before a pipeline row is finalized, but once the run is persisted the agent, telemetry, prompt, memory, evidence, provider, screenshot, and email records attach back to `pipeline_runs`.

This is the database shape behind the run-detail console. The dashboard can show the final answer and the failure path because agent runs, model calls, browser tool calls, runtime

events, prompt compilations, memory hints, streams, screenshots, providers, and email drafts are queryable as separate rows.

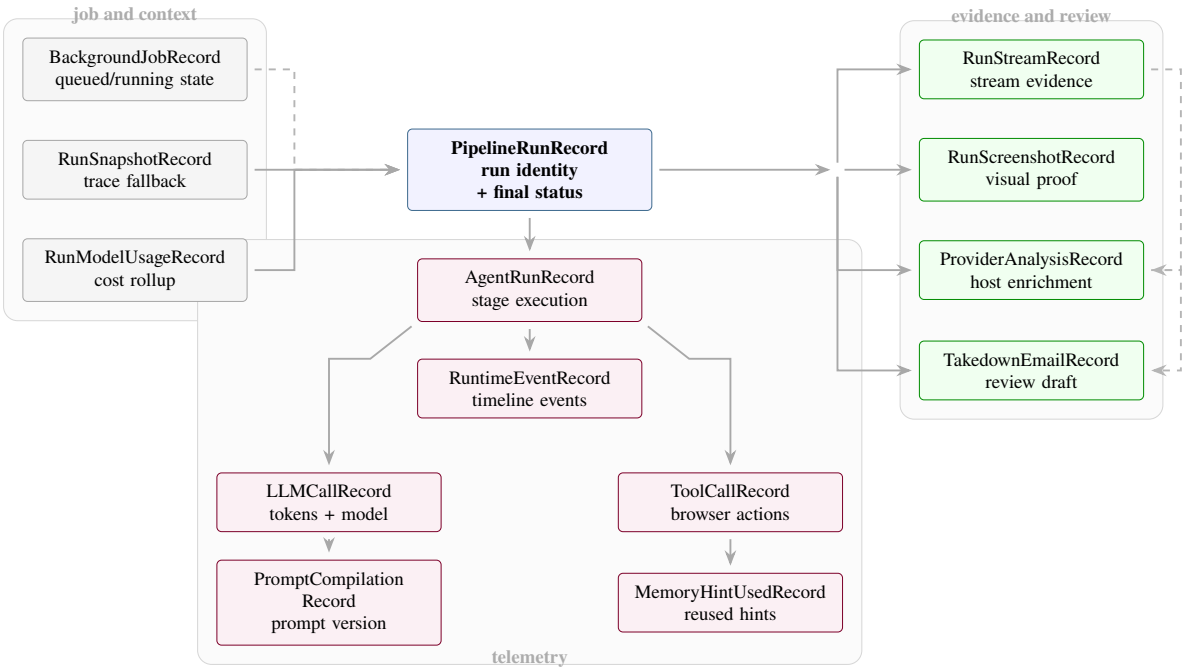


Figure 4.24: Persistence model for run evidence, telemetry, and review outputs.

Architectural element	Boundary	Reviewable output
Orchestrator state	Current URL, classification, pending hosting and embedded queues, extraction results, provider rows, emails, and final error state.	Route decisions and final `PipelineResult`.
Handoff context	Root URL, target URL, page type, route source, evidence checklist, navigation policy, and memory hints.	Bounded specialist task context.
Runtime observer	Agent lifecycle, model calls, tool calls, screenshots, status changes, cancellation, and cost counters.	Events, rollups, traces, and metrics.
Persistence layer	Background jobs, run records, agent runs, evidence rows, screenshots, streams, provider rows, and email drafts.	The data behind `GET /ui/runs/{run_id}`.

Table 4.6: Runtime data boundaries and persisted review outputs.

4.8 Chapter Summary

The architecture is a staged, reviewable evidence pipeline. The operator starts and reviews runs through the console. FastAPI controls execution and persistence. The orchestrator uses LangGraph to route between agents and to preserve handoff state [23, 24]. Each specialist agent uses LangChain and LangGraph for the model and tool loop, interprets channel evidence according to its page role, and calls profile-scoped MCP browser tools for rendered DOM context, frame inspection, screenshots, media state, and network evidence. Browser runtime controls handle profile state, fingerprint behavior, proxy selection, popup recovery, iframe recovery, and screenshot capture. Provider and email tools run only after streams exist, and storage preserves both the final answer and the failure path. The next chapter maps these architecture boundaries to the concrete implementation.

CHAPTER 5

Implementation

The implementation progressed in three major parts. The first part was an n8n workflow used as a fast exploration, validation, and crawler-backup environment. The second part was Open Web Catcher (OWC), the platform built around FastAPI, Next.js, MCP browser-tool containers, specialist agents, model/runtime management, and PostgreSQL observability. The third part was the deployment direction: the company preferred to keep n8n as the operational backup workflow while deploying the landing agent as a focused cloud job.

This split matters because the two systems did not have the same purpose. The n8n workflow was useful for testing whether agentic browsing could handle the domain at all and whether it could complement the existing crawler when static collection missed dynamic evidence. OWC was built after those lessons, with cleaner engineering boundaries, persistent traces, configurable runtime settings, and a better foundation for evaluation. The deployment work then narrowed the cloud target to the part that the company actually wanted to operationalize during the internship.

5.1 Part 1: n8n Workflow Prototype

5.1.1 Purpose of the n8n phase

The n8n implementation acted as the project playground [26]. It made it possible to test the core question quickly: can an LLM-driven browser agent inspect illegal streaming websites, navigate through landing and hosting pages, activate players, and recover useful evidence? At this stage, the priority was not a clean software architecture. The priority was proving feasibility on real, unstable websites.

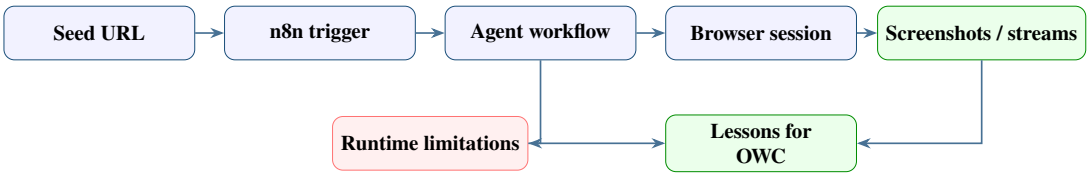


Figure 5.1: Role of the n8n workflow as a feasibility playground.

The workflow helped test agent behavior before investing in a full backend, database, and operator console. It also exposed which parts of the task were hard: browser state continuity, popup handling, stream activation, memory, runtime length, and evidence validation.

5.1.2 n8n workflow screenshots

The n8n workflow used agent-like nodes to test the same page-type responsibilities that later became specialist agents. Figure 5.2 shows the full sequential workflow used during the

prototype phase. The following figures give the specialist-agent screenshots more space so they remain readable when printed.

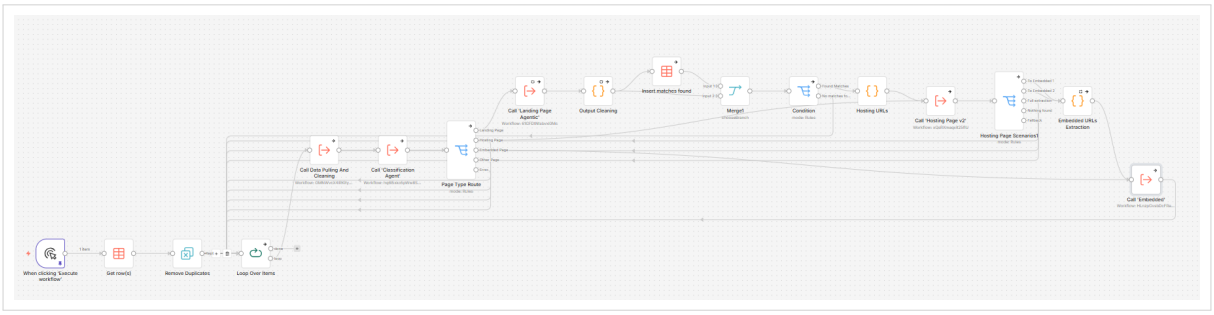
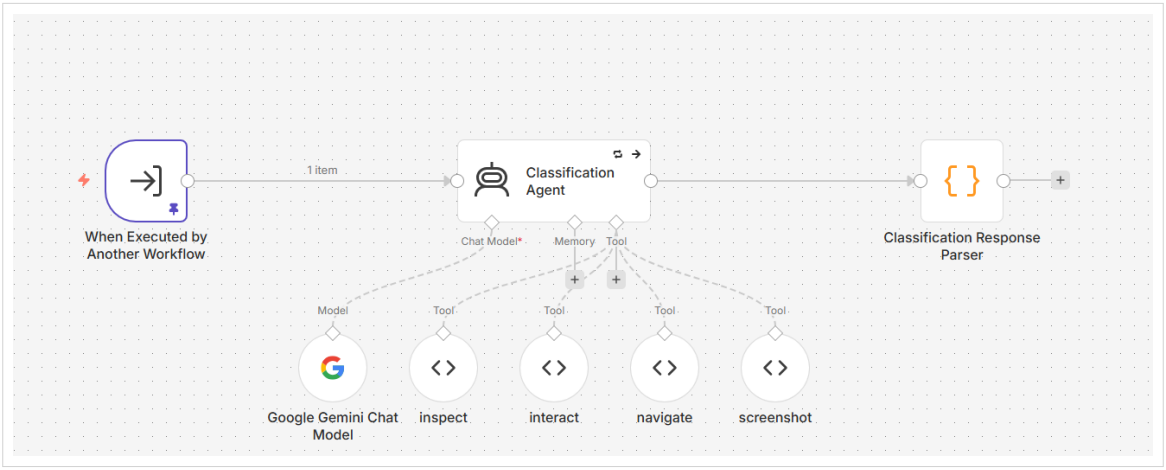
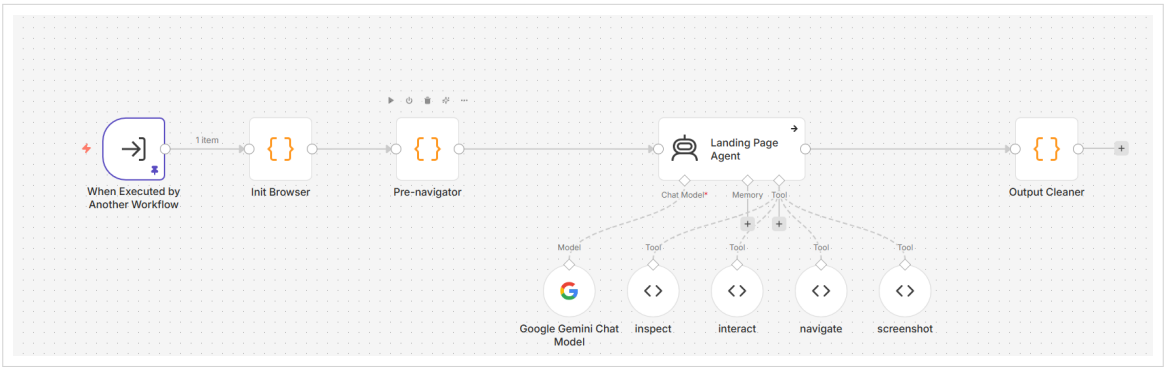


Figure 5.2: Full n8n workflow used to validate the browser-agent extraction path before OWC formalized the platform architecture.



Classification workflow

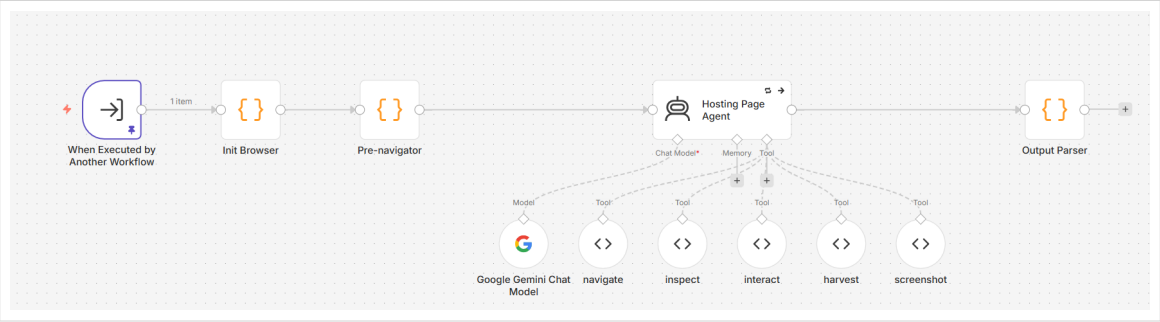


Landing-page workflow

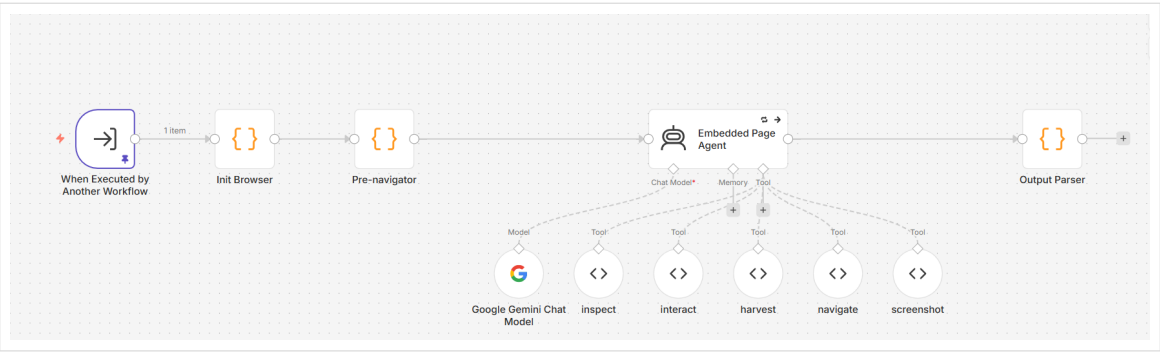
Figure 5.3: n8n classification and landing workflows, enlarged for printed review.

5.1.3 Limitations of the n8n implementation

The n8n workflow was valuable, but it reached clear limits when the task became larger than a single experimental run. These limits explain why the second implementation was necessary.



Hosting-page workflow



Embedded-player workflow

Figure 5.4: n8n hosting and embedded-player workflows, enlarged to show the browser-agent execution steps clearly.

n8n agent role	What it tested	Lesson learned
Classification	Whether a page could be categorized as landing, hosting, embedded, or other from browser evidence.	Routing must be explicit and fail-closed; wrong classification wastes the whole run.
Landing	Whether listing pages could expose match cards, tabs, and hosting links.	Generic DOM inspection was not enough; landing pages needed page-type-specific extraction.
Hosting	Whether server buttons and play controls could be activated.	Stream evidence appears only after interaction; screenshot and media-state validation are necessary.
Embedded	Whether iframe and player pages could reveal final manifests.	Nested frames require a dedicated extraction path and should not be mixed with landing-page exploration.

Table 5.1: n8n prototype agent roles and implementation lessons.

Limitation	What happened in practice	Consequence
No real evaluation layer	Runs could be inspected manually, but there was no structured evaluation harness for repeatable scoring.	Prompt changes were hard to compare objectively.
Weak memory model	The workflow did not provide a durable, domain-aware memory layer for reusable site knowledge.	Agents repeated failed actions and could not reliably reuse successful selectors or strategies.
Long runtimes	Large pages and multi-server hosting pages created long blocking executions.	Debugging became slow and failures were expensive to reproduce.
Sequential execution	The workflow behaved mostly as a sequential chain rather than parallel specialist agents.	Runs were slower than necessary, especially when multiple hosting or embedded candidates existed.
Limited development environment	n8n is a workflow editor, not a normal source controlled development environment.	Workflows had to be exported as JSON files to preserve progress and compare versions.
Fragile Puppeteer integration	Puppeteer code inside n8n nodes was brittle and tools frequently broke when browser state changed.	Browser automation needed a dedicated tool service with stable sessions.
Isolated code execution	Code executions were isolated, so browser instances could be lost between steps. A `puppeteer.connect` workaround was needed to reconnect to a running browser.	Browser continuity became a workaround instead of a clean runtime contract.
High memory usage	Long browser sessions, page artifacts, and workflow state created large memory pressure.	The prototype was not comfortable for long or batch-oriented extraction runs.
Hard failure analysis	Errors were visible, but not stored as normalized events, LLM calls, tool calls, screenshots, and costs.	Post-mortem review required manual reconstruction.

Table 5.2: n8n prototype limitations and engineering consequences.

5.2 Part 2: Open Web Catcher Platform

OWC keeps the useful extraction logic from the n8n phase but moves it into explicit software components. It is organized into five main implementation areas: FastAPI, Next.js, MCP browser containers, agents and model runtime, and PostgreSQL.

5.2.1 FastAPI Backend

The FastAPI backend is the control plane of OWC [21]. It starts runs, checks runtime readiness, streams or returns run state, exposes settings, stores observability records, and gives the frontend a stable API surface.

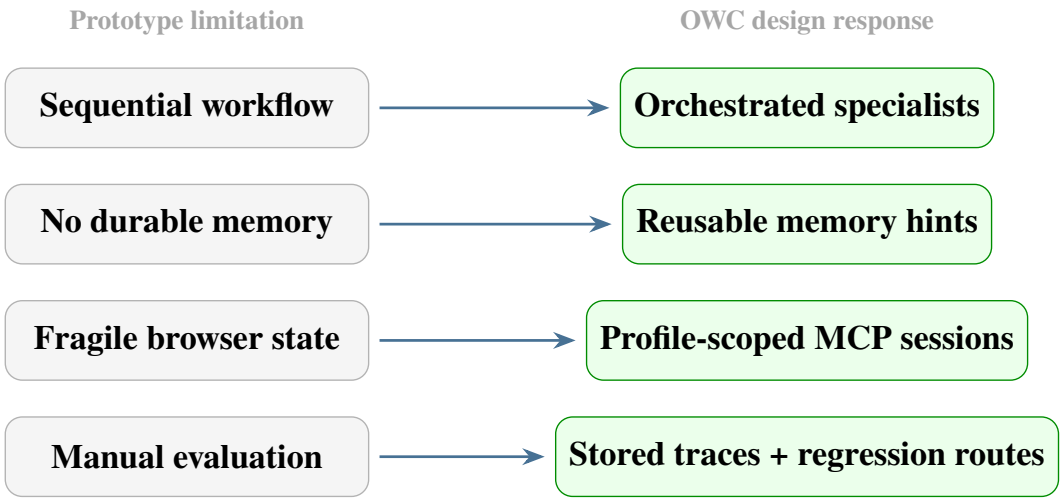


Figure 5.5: How n8n limitations shaped the Open Web Catcher implementation.

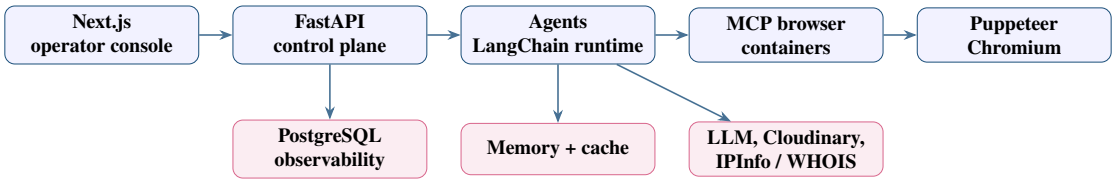


Figure 5.6: Five implementation areas of the Open Web Catcher platform.

Backend area	Responsibility	Why it matters
Runtime health	Check browser, MCP, settings, and launch readiness.	Failed preflight is clearer than a silently broken agent run.
Run execution	Start single URL runs and dataset-backed runs.	Keeps workflow launch separate from UI rendering.
Run inspection	Return trace, screenshots, streams, provider data, model usage, and errors.	Makes each run auditable after execution.
Configuration	Persist model, provider, pricing, browser, and tool settings.	Allows controlled experimentation without editing code every time.
MCP tool diagnostics	Expose profile-specific tool status and logged tool calls.	Separates browser/tool failures from prompt failures.
Evaluation routes	Run or review batches and reliability checks.	Provides the foundation that the n8n prototype lacked.

Table 5.3: Backend API responsibilities in OWC.

The backend also uses lightweight caching for frequently polled read endpoints. This reduces repeated database work when the frontend refreshes dashboards or run summaries.

5.2.2 Next.js Operator Console

The Next.js frontend is not just a dashboard [22]. It is the operator console for OWC: launching runs, following execution, reviewing evidence, and changing runtime configuration.

Frontend surface	What it shows or controls	Main backend dependency
Dashboard	Overview, cost, token, provider, tool, and agent telemetry.	Dashboard aggregation endpoints and persisted observability tables.
Live Pipeline	Workflow and single-agent launch with runtime preflight.	Workflow launch, health, and browser/MCP readiness endpoints.
View Results	Website dataset, batch execution, per-site status, and run history.	Dataset, batch, and run repositories.
Provider Results	Stream URL enrichment, IP resolution, provider distribution, country distribution, and manual lookup.	Provider-analysis records and lookup services.
Settings	Gemini model assignments, cache/runtime controls, browser settings, display preferences, API key status, notifications, and MCP tool enablement.	Configuration endpoints and runtime settings persistence.

Table 5.4: Current Next.js operator-console surfaces and backend dependencies.

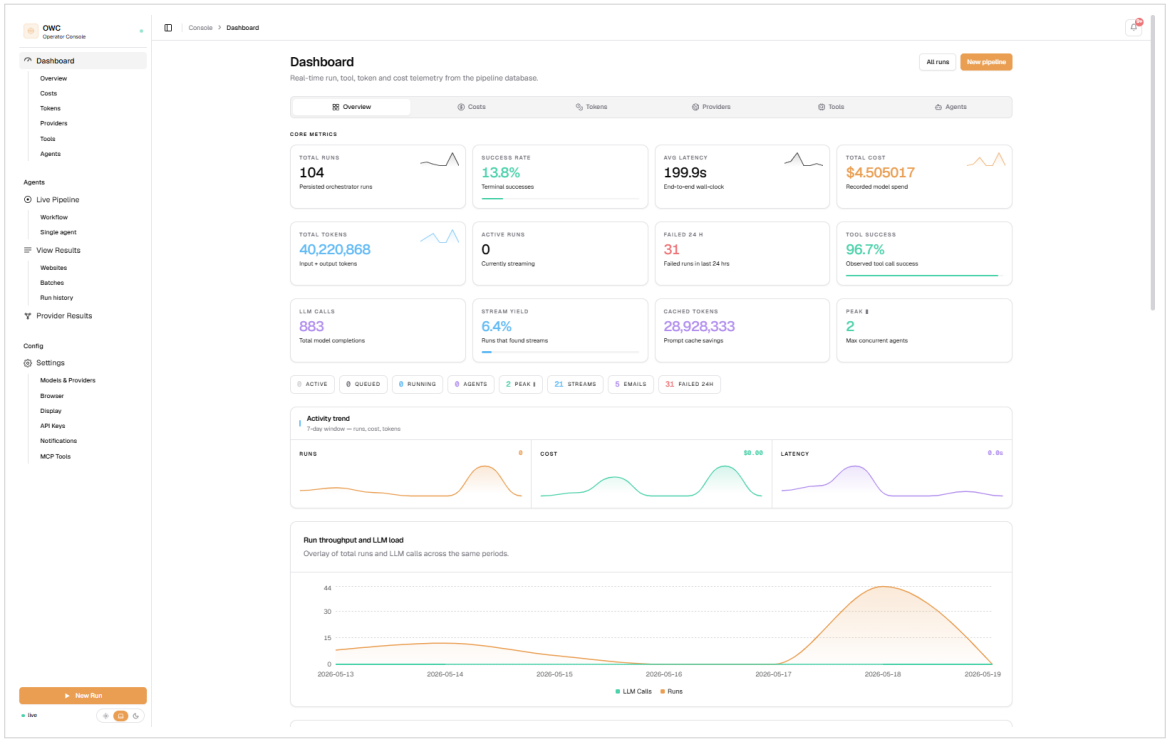
The design goal was operational clarity. A failed launch should show why it failed. A completed run should show what the agent actually did, which tools it called, what evidence it found, and where the model or browser runtime failed.

5.2.3 MCP Browser Containers and Puppeteer Tools

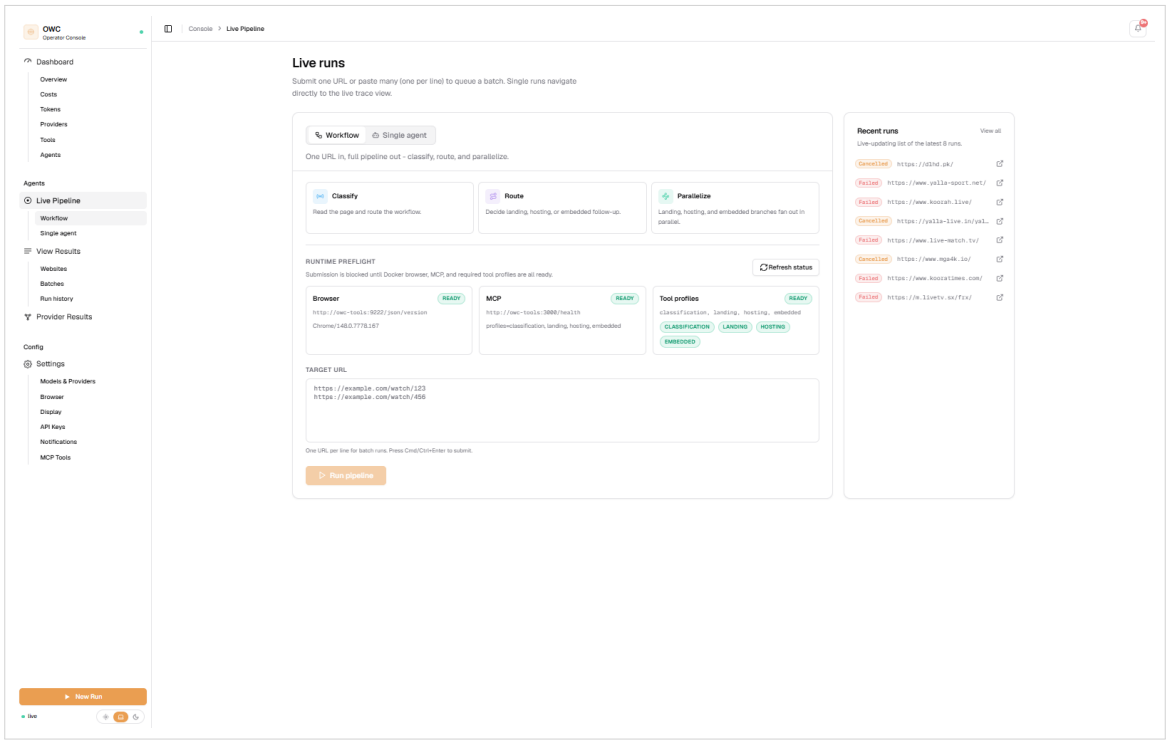
The browser tooling was extracted from the n8n code-node style into dedicated MCP browser services [25]. Each agent profile receives a filtered tool surface through an MCP endpoint. This reduces tool-selection noise and keeps browser session ownership outside the LLM prompt. The engine comparison below is based on the official Playwright and Puppeteer references and the project's integration constraints [13, 12, 18].

5.2.3.1 Tool registry and profile filtering

The MCP server exposes a larger internal tool registry than any one agent should use. During a run, OWC binds only the profile-specific subset required by the current stage. Classification receives broad page inspection and screenshot access. Landing receives tools for listing-page context, candidate extraction, reveal actions, scrolling, and element details. Hosting receives tools for server controls, playback activation, popup recovery, media state, and stream harvesting.

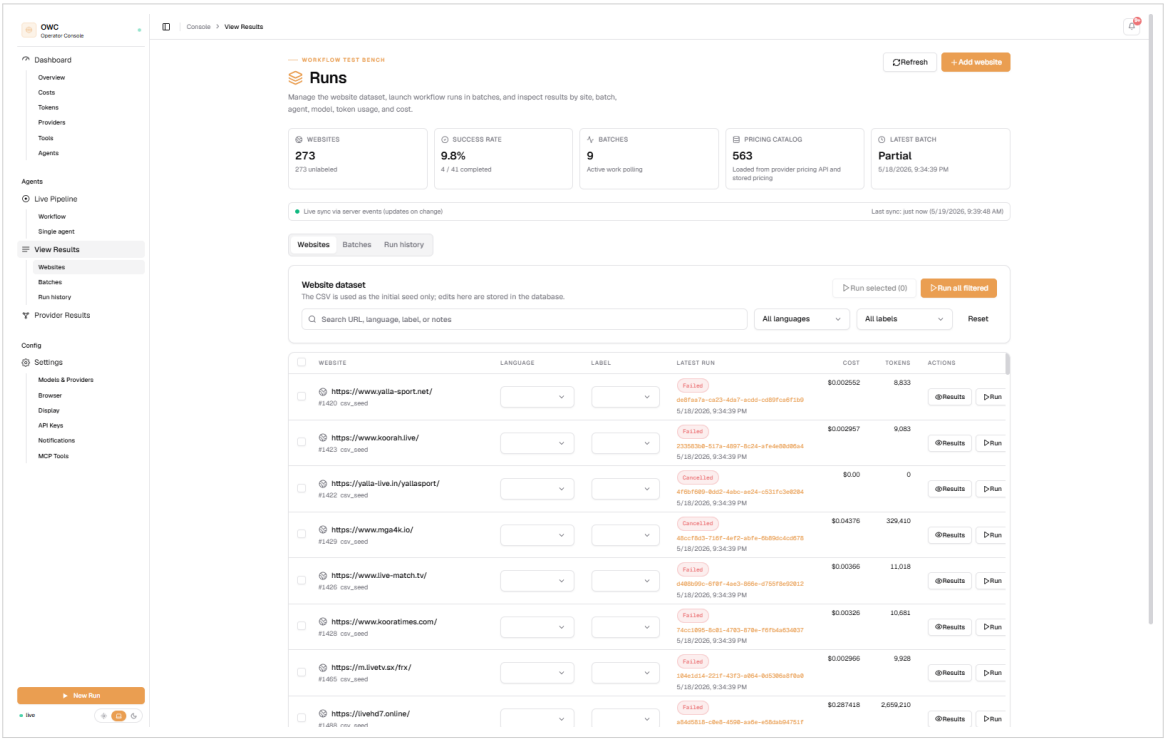


Dashboard telemetry

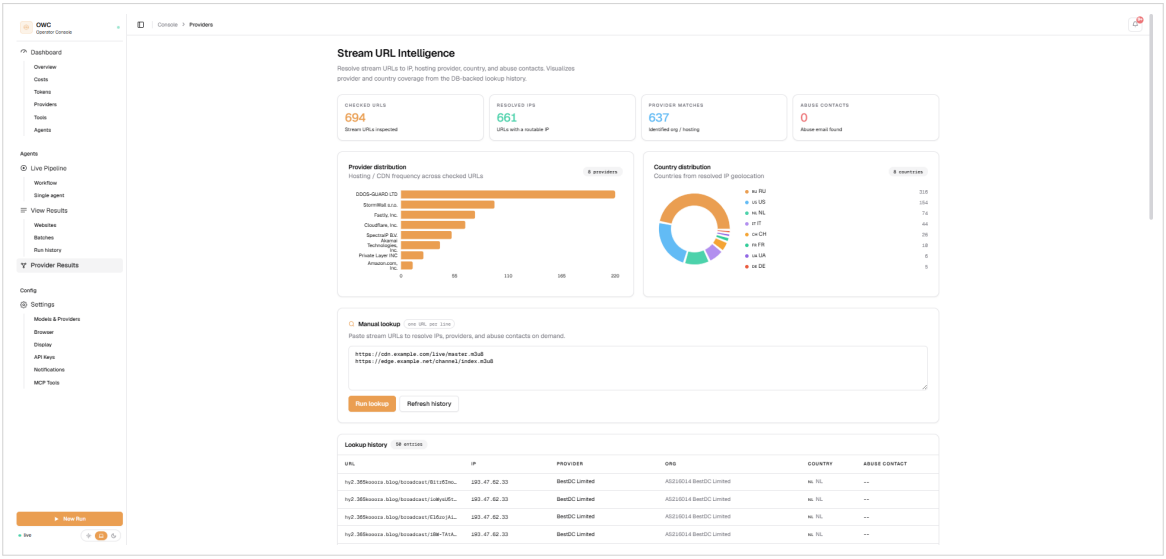


Live workflow launcher

Figure 5.7: Main OWC operator-console screens for dashboard telemetry and live run launch.

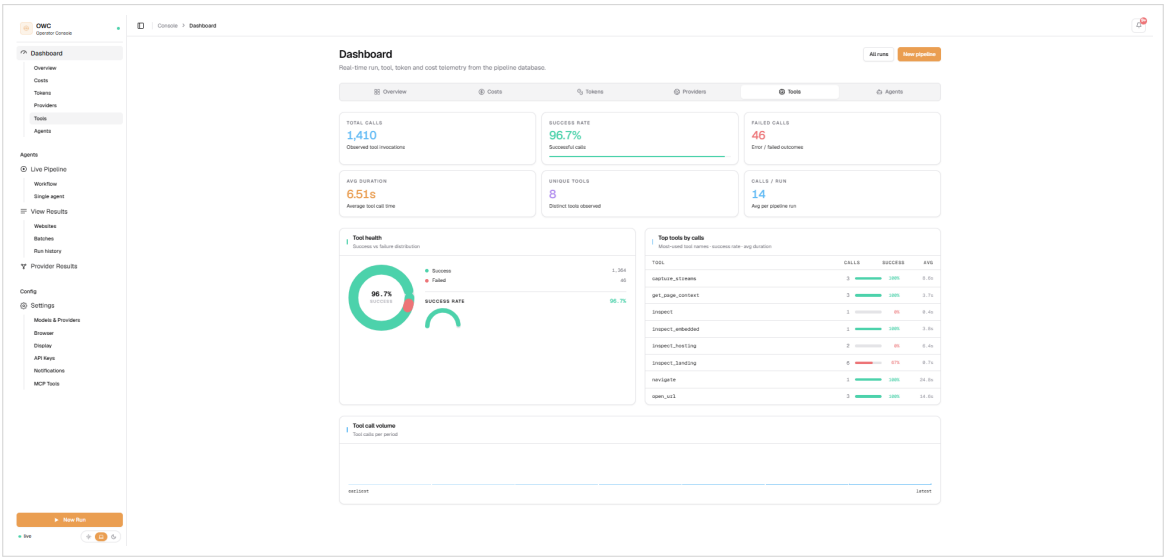


Run and website review

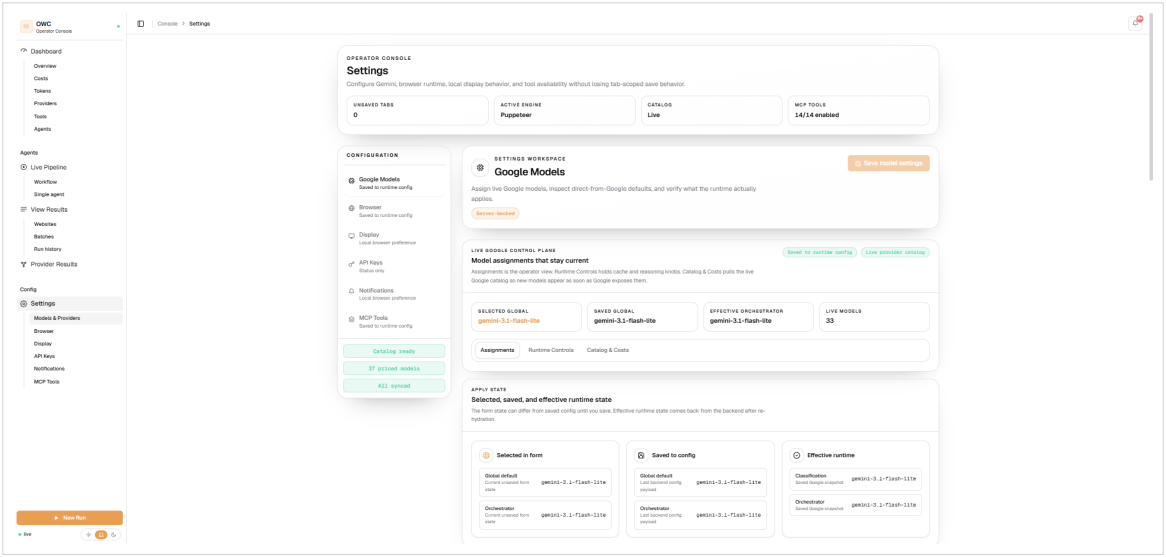


Provider enrichment review

Figure 5.8: OWC review screens for run results and provider enrichment, enlarged so tables remain readable in print.



Tool telemetry



Model and runtime settings

Figure 5.9: OWC tool telemetry and runtime configuration screens, enlarged for printed review.

Browser engine	Strengths observed	Project decision
Playwright	Strong context isolation, good frame and browser-context APIs, useful for comparing behavior and validating browser policies.	Tested and kept as a useful reference path, but not the primary runtime.
Puppeteer	Direct Chrome DevTools Protocol fit, simpler integration with the existing tool registry, stable enough for the profiled MCP tool server.	Chosen as the main browser engine for the implemented agent prompts and tool runtime.

Table 5.5: Browser automation engine comparison for the MCP tool layer.

Tool group	Purpose
Navigation tools	Open URLs, go back, wait for page state, and recover from navigation drift.
Inspection tools	Read page context, landing cards, hosting server controls, embedded player state, DOM elements, and frame trees.
Interaction tools	Click by selector, text, XPath, checkbox/radio/select, coordinates, play media, type input, scroll, and swipe.
Evidence tools	Capture screenshots, media state, stream requests, player signals, and harvested `m3u8/mpd/mp4` candidates.
Memory tools	Read or update reusable site hints from the memory layer.

Table 5.6: Puppeteer MCP tool groups and responsibilities.

Embedded receives frame-focused tools that keep the run inside the assigned iframe or player URL. This filtering prevents a landing task from drifting into provider analysis or a hosting task from rewriting the whole route.

5.2.3.2 Rendered DOM context retrieval

The implementation uses Puppeteer because the useful evidence is usually in the rendered browser state rather than the initial HTML response. Context tools execute inside the active Chrome page, read visible text and DOM structure, group anchors and buttons, inspect iframe trees, and return compact JSON observations. The agent first asks for a broad page context, then asks for a scoped element or frame detail before acting. This two-step pattern reduced repeated full-page dumps and made long runs cheaper to review because the trace shows why a selector, server tab, iframe, or candidate URL was chosen.

5.2.3.3 Puppeteer actions and evidence feedback

Action tools are implemented as browser-state changes followed by explicit evidence checks. A click, scroll, text input, server switch, play action, or popup recovery is not considered useful by itself. The agent must inspect the page again, capture a screenshot when the visual state matters, read media state when a player is involved, or harvest network candidates after playback. This is why the tool trace alternates between observation, action, and verification instead of showing one large scripted Puppeteer procedure.

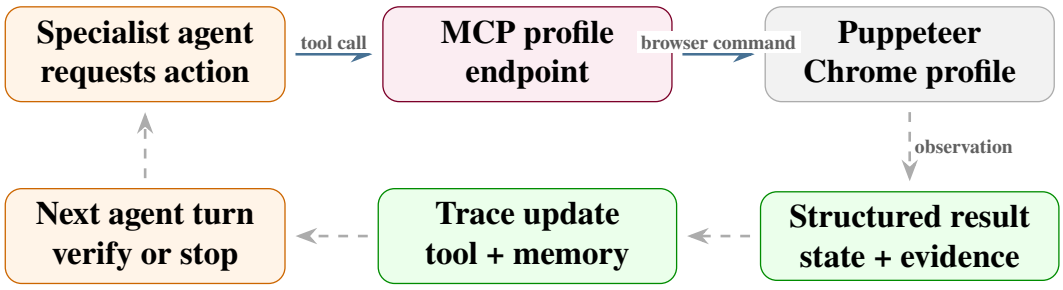


Figure 5.10: Browser tool execution loop through MCP and Puppeteer.

5.2.4 Agents, Models, Memory, and Prompts

OWC uses specialist agents rather than one general browser agent. Each specialist receives a role-specific prompt, a profile-specific tool set, and a bounded execution budget.

Agent	Objective	Main tools used
Classification	Decide whether the seed is landing, hosting, embedded, or other, while preserving obvious channel hints.	Generic inspection, screenshot, frame tree, memory lookup.
Landing	Discover match cards, listing tabs, pagination, and hosting candidates with match and channel context.	Landing inspect, element queries, click/scroll, screenshot, memory tools.
Hosting	Activate server controls, verify player state, and extract streams or embedded fallbacks while checking channel/player labels.	Hosting inspect, click/play tools, media state, frame tree, harvest, screenshot.
Embedded	Work only on embedded/player URLs and harvest final stream evidence.	Embedded inspect, play/media tools, harvest, screenshot, frame diagnostics, iframe-local text checks.

Table 5.7: Specialist agent objectives and assigned tool surfaces.

5.2.4.1 Design constraints implemented in the agent runtime

The agent runtime was shaped by constraints that became clear during prototyping. These constraints are implementation concerns, not only methodology notes, because each one changed how the agents, prompts, tools, memory, and stopping rules were built.

5.2.4.2 Prompt evolution in the implementation

The prompts evolved from linear intent prompts into evidence-driven loops. The early prompt asked the model to solve the page in one pass. That worked on simple pages, but it failed on ambiguous layouts because the agent would stop before opening tabs, activating players, or verifying stream generation. The implemented prompt pattern instead forces each specialist to observe, reason, act, and validate before returning a result.

5.2.4.3 Agent design principles

The resulting agent design follows seven practical principles:

- classify the page role before deep extraction;
- separate operational n8n validation from the OWC platform architecture;
- ground every conclusion in browser evidence rather than textual guessing;
- preserve traces, screenshots, and structured outputs for each critical step;
- use specialist agents and explicit fallbacks instead of one unconstrained agent;
- optimize reliability and cost together through caching, budgets, and model routing;

Constraint	Runtime effect	Implementation response
Changing layouts	Selectors and visible labels differ across languages, clones, and match-list layouts.	Agents use broad DOM context first, then scoped element details and page-type-specific tools.
Nested iframe players	The playable media source may not exist in the top-level page.	Hosting agents can hand off embedded URLs; embedded agents inspect frame trees and player-only pages.
Popups and ad traps	Clicks can open new tabs, steal focus, or hide the original player.	Browser tools restore focus, close popup tabs, and retry with safer JavaScript interactions.
Delayed stream generation	Stream manifests often appear only after a play click or server switch.	Prompts require media-state or screenshot verification before harvesting network evidence.
Cost and context growth	Repeated DOM observations, screenshots, and retries can make runs expensive.	The runtime tracks token usage, uses caching, and enforces bounded tool budgets.
Evidence isolation	The reference system must not depend on company production data.	Persistence is limited to traces, metrics, screenshots, tool calls, model usage, and structured outputs.

Table 5.8: Runtime constraints and their implementation responses.

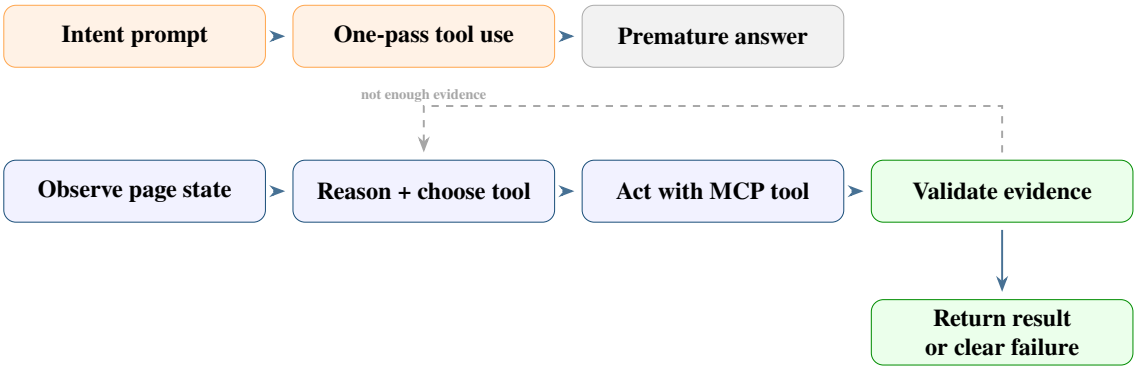


Figure 5.11: Prompt evolution from one-pass intent prompts to evidence-driven agent loops.

- keep deployment loosely coupled across API services, browser workers, observability storage, and queue-based execution.

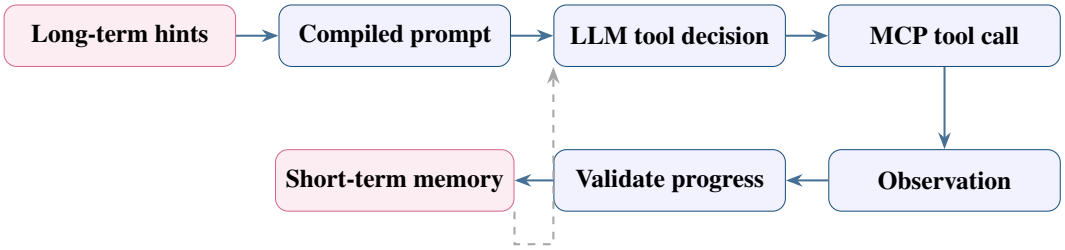


Figure 5.12: Agent reasoning loop with memory and tool feedback.

LangChain is used as the tool-binding and message-loop layer [23]. The implementation does not let the model freely access every browser capability. Instead, the MCP client loads only the tool profile for the current agent. The shared loop records tool calls, model calls, token usage, cache behavior, and stop reasons.

5.2.4.4 Model behavior, token usage, and rate limits

The model layer had to be treated as an operational dependency, not only as a prompting detail. Long browser pages, repeated DOM observations, screenshots, tool traces, and failed retries can grow the context quickly. The implementation therefore tracks context windows, input tokens, output tokens, cached tokens, cache writes, and estimated cost.

Gemini rate limits are evaluated across RPM, TPM, and RPD. Google's current provider reference defines these as requests per minute, input tokens per minute, and requests per day, and states that limits depend on model and usage tier and can change in AI Studio [9]. The implementation therefore treats rate limits as configuration and observability concerns rather than hard-coded constants.

Model concern	Implementation handling	Why it matters
Context window	Store and display model context-window metadata where available.	Prevents long runs from silently approaching context exhaustion.
Token usage	Track input, output, cached input, and cache-write tokens.	Makes cost and prompt growth visible per run.
Tool calling	Bind only profile-specific tools through LangChain and MCP.	Reduces incorrect tool choice and keeps browser actions controlled.
Rate limits	Treat RPM, TPM, and RPD as provider limits to monitor and respect.	Avoids confusing quota failures with browser or prompt failures.
Retries	Apply bounded LLM retry attempts and delays per agent profile.	Recovers transient provider errors without infinite loops.
Caching	Use prompt/provider cache and tool-result cache where safe.	Reduces repeated tokens and repeated browser observations.

Table 5.9: Model-runtime concerns handled by the agent platform.

5.2.4.5 Prompt engineering and stopping behavior

The prompts evolved from broad goals into operational procedures. Each prompt defines the objective, the evidence required, which page ownership rules must be respected, and when the agent should stop. The strongest prompt pattern was an evidence-based loop: observe, state the current hypothesis, choose one action, verify the result, then either continue or return a structured failure.

Prompt design rule	Implementation effect
Explicit page ownership	Hosting agents stay on hosting pages; embedded agents stay on embedded/player URLs.
Evidence before harvest	Agents confirm player state or screenshot state before claiming stream extraction.
Bounded tool budget	Long runs stop with a clear partial result instead of looping indefinitely.
Structured output contract	Results can be stored, evaluated, and rendered in the UI without manual parsing.
Memory hints as soft guidance	Prior site knowledge helps, but does not replace current evidence.

Table 5.10: Prompt engineering rules enforced by the specialist agents.

5.2.5 PostgreSQL Database and Observability

The PostgreSQL database is used mainly for observability, review, evaluation, and configuration. It is not the product database of the external organization. Its purpose in this project is to preserve what happened during a run so that failures and successes can be inspected later.

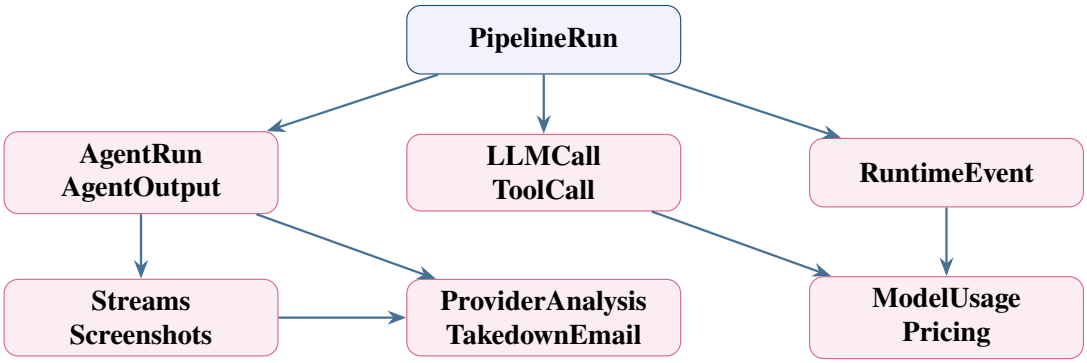


Figure 5.13: Simplified database relationship map for run review.

5.3 Part 3: Deployment

The company preferred to keep n8n as the operational backup workflow around the existing crawler. For that reason, the cloud deployment did not attempt to replace the whole workflow

Record group	Tables / records	Status in the project
Core run trace	Pipeline runs, agent runs, runtime events, LLM calls, tool calls.	Central to reviewing and debugging runs.
Evidence	Run streams, screenshots, provider analyses, takedown emails.	Central to the evidence bundle.
Model and prompt metadata	Prompt versions, prompt compilations, model usage, pricing configs.	Important for cost, prompt iteration, and reproducibility.
Memory	Memory entries and memory hints used.	Used to connect cross-run knowledge with current agent decisions.
Datasets and batches	Dataset sites, dataset batches, dataset site runs.	Useful for evaluation/batch execution, but not always used in every run.
Operational support	Run decisions, run tasks, tool-call diagnostics, snapshots.	Support/diagnostic records; some remain secondary rather than core report outputs.

Table 5.11: PostgreSQL observability records and their implementation role.

with the complete OWC platform. Instead, the deployment focused on one validated unit: the landing-page agent. The deployment shape follows the practical company decision: target URLs, which are usually landing pages, arrive through an Azure Service Bus queue named `agent_tasks`; one Azure Container Apps Job container owns n8n, the landing-agent worker, the Puppeteer MCP server, and Chrome instances; and the extracted matched URLs plus metadata are returned through a second Azure Service Bus queue named `agent_results` [27, 28].

This flow keeps the crawler, n8n, and the AI browser agent loosely coupled. The crawler can continue broad discovery. n8n can decide when a page needs the backup landing agent. The container can perform the expensive rendered-page work outside a synchronous web request. The result queue can then return candidate hosting URLs, channel hints, screenshots, trace metadata, and failure reasons to the operational workflow.

This deployment choice matches the internship scope. OWC shows how the full platform can be structured, but the company-side deployment remained focused: n8n stayed in place as the backup orchestrator, while the landing-page agent was isolated as a cloud-executable single-container unit.

5.4 Implementation Summary

The implementation moved from a useful but limited n8n prototype to the structured OWC platform. The n8n phase validated the feasibility of agentic browser extraction and exposed the main runtime problems. OWC addressed those problems with explicit backend routes, a reviewable frontend, profiled MCP browser tools, specialist agents, memory and caching, model-usage tracking, and PostgreSQL observability.

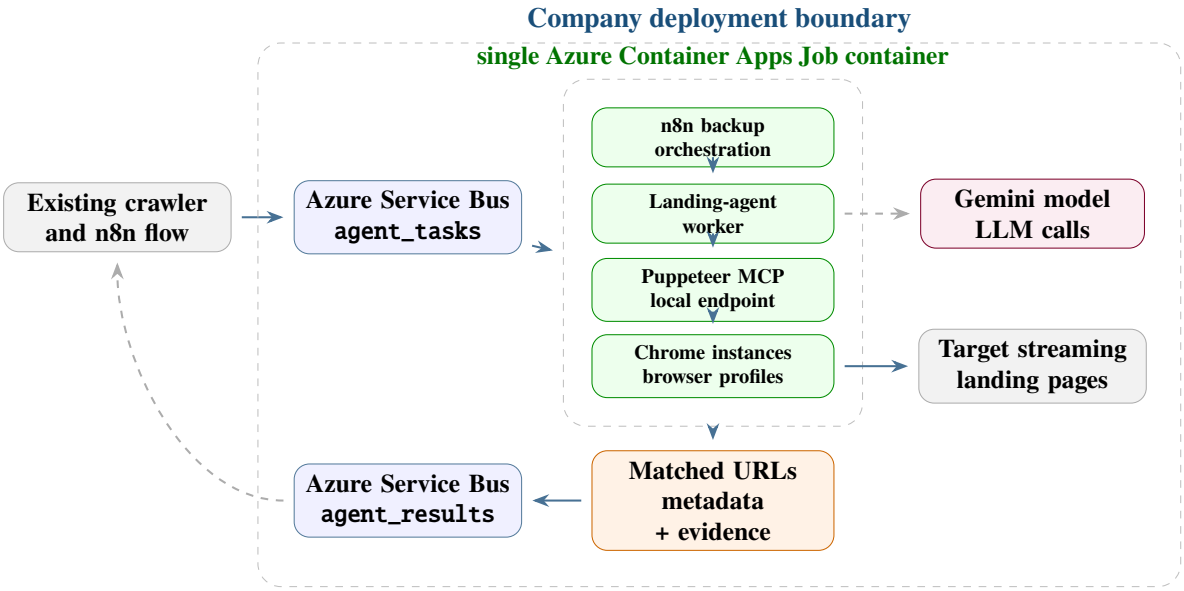


Figure 5.14: Company deployment flow using `agent_tasks`, a single n8n/Puppeteer landing-agent container, and `agent_results`.

Deployment element	Role	Scope during internship
n8n workflow	Keeps the company-preferred orchestration layer.	Main operational workflow remained in n8n.
Azure Service Bus	Decouples job submission from execution.	Used to pass landing-page jobs to the container worker.
Container Apps Job	Runs one containerized agent unit when a message arrives.	Single image for n8n, the landing-agent worker, Puppeteer MCP, and Chrome.
Puppeteer runtime	Opens the page and extracts match/host candidates through the local MCP endpoint.	Runs inside the same job container as the landing agent.
Output queue	Publishes matched URLs, metadata, screenshots, logs, and failure reasons to `agent_results`.	Used for review and integration back into the workflow.

Table 5.12: Landing-agent deployment scope and responsibilities.

The result is not only an extraction script. It is a reviewable agent platform where failures can be diagnosed, prompts can be evaluated, tool behavior can be tested independently, and evidence can be inspected after the browser run ends.

CHAPTER 6

Evaluation and Testing

This chapter evaluates the system as a browser-based evidence collection workflow. The goal is not only to classify a page correctly. A website is considered working only when the run reaches usable evidence: the page is classified, match or host candidates are extracted when the page is a landing page, the agent reaches a playable page, at least one networking or embedded media link is returned, and a screenshot shows the player state.

The evaluation is organized around three main points. First, agent evaluation measures whether the specialist agents can keep old websites working while new websites are added. Second, model evaluation compares model behavior, KPI stability, cache usage, and cost. Third, tool evaluation checks whether the browser tools extract the evidence they are supposed to extract. All three use the same regression idea: a change is accepted only if it improves new cases without breaking websites that previously worked.

6.1 Agent Evaluation and Website Regression

The main evaluation set is a batch dataset of more than 200 streaming websites. The dataset intentionally mixes languages, layouts, player types, redirect styles, and similar website families. This prevents the agents from being tuned only for one English layout or one player implementation. The same websites are rerun across prompt variations, model settings, and tool changes. Tests were run multiple times to reduce the risk of accepting a result caused by a lucky model response or temporary website state.

6.1.1 Test Protocol

The batch test follows the same sequence for each prompt, model, or tool change. First, the website list is loaded from the dataset. Each row contains a target URL and optional notes about language, layout, or expected page family. Second, the pipeline starts with classification and routes the URL to the proper specialist agent. Third, the specialist agents try to collect evidence: landing agents extract matches and host candidates, hosting agents activate players and switch servers, and embedded agents inspect iframes and harvest network media evidence. Fourth, the run output is checked against the acceptance rule. Finally, results are compared with previous runs for the same website.

This means the percentages in this chapter are not abstract model scores. They are website-batch scores. If a website is classified but no match, media link, or useful screenshot is returned, it does not count as a working website. Repetition is important because LLM decisions and streaming websites are both variable: a prompt or model change is trusted only when it survives repeated runs without losing previously working websites.

Test type	What is executed	What is recorded
Agent batch test	Run the complete pipeline on the 200+ website dataset.	Per-website pass/fail, page type, extracted matches, stream or embedded links, screenshot evidence, and failure reason.
Prompt regression test	Rerun the same dataset after changing agent instructions.	Websites newly fixed, websites still working, and websites that regressed.
Model comparison test	Rerun representative batches with the same prompt/tool setup but different model choices.	Website success rate, consistency across repeated runs, tokens, cache hit ratio, cost, and rate-limit behavior.
Tool regression test	Rerun the batch after changing browser tools or tool descriptions.	Whether inspect, navigate, interact, screenshot, harvest, and provider tools extract their intended evidence.
Manual evidence review	Inspect final traces, screenshots, stream URLs, and provider results for selected pass/fail cases.	Whether the automated pass/fail label is defensible and whether the failure belongs to prompt, model, tool, or website behavior.

Table 6.1: Evaluation test matrix executed over the website regression batch.

6.1.2 Batch Acceptance Rule

A website batch row is counted as successful only when the investigation returns enough evidence to be useful. The acceptance rule is stricter than a simple "run completed" status.

Checkpoint	Website is counted as working when
Page classification	The page type is correctly routed to landing, hosting, embedded, or other.
Match extraction	For landing pages, visible matches or event cards are extracted with usable links or host candidates.
Navigation	The agent reaches the next useful page instead of stopping at the first listing or ad page.
Player activation	The agent interacts with the player or server controls until a visible playback state, media state, or stream attempt is observed.
Network evidence	The run returns at least one useful network, embedded, manifest, or media URL candidate.
Screenshot evidence	A screenshot supports the final claim, preferably showing the player or the failure state that blocked playback.

Table 6.2: Website-level acceptance criteria for regression success.

The batch score is therefore calculated per website, not per tool call:

$$WebsiteSuccessRate = \frac{working\_websites}{evaluated\_websites}$$

$$RegressionRetention = \frac{previously\_working\_websites\_still\_working}{previously\_working\_websites}$$

The practical objective is to raise support for new websites while keeping *RegressionRetention* high. A prompt or tool change that supports a new player but breaks older working websites is treated as a regression.

6.1.3 Agent Use Cases

Each specialist agent is evaluated on the part of the investigation it owns. This makes failures easier to diagnose: a run can fail because the classifier routed incorrectly, the landing agent missed host candidates, the hosting agent failed to activate a server, or the embedded agent harvested too early.

Agent	Evaluated use cases	Success signal in the batch
Classification	Landing, hosting, embedded, unknown/other pages, redirects, blocked pages.	Correct route is selected and the next specialist receives the right task.
Landing page	Match cards, schedule lists, language-specific labels, tabs, pagination, and duplicate links.	Matches and host candidates are extracted without unrelated navigation noise.
Hosting page	Server buttons, play overlays, popups, ad tabs, redirects, and embedded handoff.	A server/player path is activated and a stream candidate or embedded player handoff is returned.
Embedded page	Iframes, player-only URLs, delayed manifests, tokenized stream URLs, and video state checks.	Network/media evidence is extracted and supported by a screenshot or clear failure reason.
Provider analysis	IPInfo, RDAP, WHOIS, Cloudflare-like providers, and abuse-contact metadata.	Extracted stream or host evidence can be enriched when provider data is available.

Table 6.3: Agent use cases and website-level success signals.

6.1.4 Prompt Regression Results

Prompt evaluation uses the same website batch for every prompt generation. The value in the final column is the percentage of websites in the batch that still met the acceptance rule after that prompt change. It is not an average number of tool calls.

The important trend is that v1 was not truly efficient; it simply stopped too early. The ReAct prompt improved behavior because it forced the agent to observe, act, and verify instead of guessing from the first page. Later versions improved the hard cases: popups, server switching, media-state checks, tokenized streams, and early exit after sufficient evidence was collected.

Version	Change tested	Regression focus	Batch websites working
v1	Intent-based prompt	Early stop, weak tab navigation, premature harvest.	38%
v2	ReAct loop and tab-check instruction	Navigation recall and screenshot-grounded decisions.	68%
v3	Popup handler and JavaScript click fallback	Popup handling and server activation after ad traps.	74%
v4	Mandatory media-state check before harvest	Player activation and premature harvest prevention.	81%
v5	Tool budget cap and early success exit	Cost reduction without losing previously working websites.	83%

Table 6.4: Prompt regression history and batch-level behavioral focus.

Metric	Intent v1	ReAct v2	Current v3/v5
Page-type routing accuracy	91%	93%	95%
Host-candidate recall	40%	85%	89%
Player activation success	0%	70%	90%
Popup-interference recovery	20%	55%	78%
Tokenized-stream recovery	25%	40%	85%
Manifest recovery on AlbaPlayer case	0%	70%	90%
Manifest recovery on tokenized Clappr case	–	40%	85%

Table 6.5: Use-case KPI results measured on the website regression batch.

6.1.5 Dashboard Snapshot

The dashboard is used to inspect what the batch execution produced. It is not only a visual summary; it is the place where run, token, cost, provider, tool, and agent telemetry are checked after experiments. The screenshots used for this report show a larger dashboard snapshot than the earlier local test database: model costs, tokens, provider lookups, and tool health are all visible.

These values complement the website-batch scores. The batch score says whether a website worked. The dashboard explains why a run was expensive, whether caching helped, which models were used, which tools failed, and whether extracted stream URLs could be enriched with provider information.

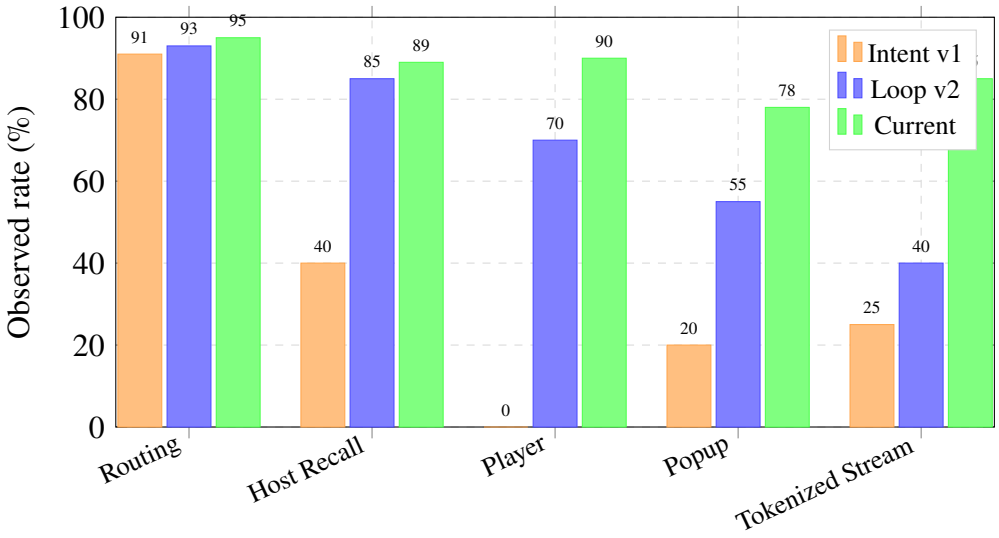


Figure 6.1: Regression-batch improvement across the main browser-agent KPIs.

Dashboard area	What it validates	Observed value
Cost telemetry	Model-spend aggregation and average cost per run.	Total cost: $4.505017; average cost/run: $0.047926.
LLM calls	Whether the batch persisted model invocations.	883 calls; 8 calls/run.
Token telemetry	Whether input, cached input, and output tokens are separated.	Total: 40,220,868; output: 454,914.
Cache behavior	Whether repeated prompt/tool context is being reused.	Cached input: 28,928,333; cache hit rate: 40.0%.
Provider intelligence	Whether extracted stream URLs can be resolved to infrastructure metadata.	694 checked URLs, 661 resolved IPs, 637 provider matches.
Tool health	Whether browser tool execution is stable during runs.	1,410 calls, 96.7% success, 46 failed calls.

Table 6.6: Dashboard telemetry values used as operational evidence.

6.2 Model Evaluation, KPI Selection, and Cache

Model evaluation uses the same 200+ website batch and the same prompt/tool configuration for each model candidate. This keeps the comparison fair: the metric should reflect model behavior, not a different browser policy or a different prompt. Each model is evaluated on quality, consistency, cost, cache behavior, and rate-limit practicality.

6.2.1 KPI Formulas

The evaluation uses website-level KPI formulas. Let N be the number of evaluated websites in the batch, W the number of websites that satisfy the acceptance rule, C the number correctly classified, H the number where valid host candidates are found, P the number where the player is activated, and E the number where media/network evidence is returned.

KPI	Formula and meaning
Website success rate	W/N. Main regression metric: the percentage of sites that still work end-to-end.
Classification accuracy	C/N. Measures whether the correct specialist route is selected.
Host-candidate recall	valid host candidates found divided by manually expected host candidates for landing pages.
Player activation rate	$P/$ activation attempts. A success requires playback state, media state, or screenshot-supported stream attempt.
Evidence extraction rate	E/N. Measures websites returning at least one network, manifest, embedded, or media URL candidate.
Regression retention	previously working websites still working divided by previously working websites.
Consistency rate	websites with the same pass/fail result across repeated runs divided by repeated websites.
Cache hit ratio	$cached\_input\_tokens/(cached\_input\_tokens + new\_input\_tokens)$.
Cost per working website	total estimated model cost divided by websites that satisfy the acceptance rule.

Table 6.7: KPI formulas for model, agent, and website-batch evaluation.

6.2.2 Model Selection Criteria

A stronger model is not selected only because it produces longer reasoning. It must improve the website-level score while staying operationally usable.

6.2.3 LLM Model Comparison

Several model families were tested before selecting the final runtime model. The final choice was `google_genai::gemini-3.1-flash-lite`. The decision was not based on one benchmark number. It came from batch behavior: the model had to keep enough context for

Dimension	What is measured	Why it affects selection
Accuracy	Website success rate, classification accuracy, evidence extraction rate.	The selected model must increase successful sites, not just produce confident text.
Consistency	Same website and prompt run multiple times.	Streaming sites are noisy; the model should not randomly lose working websites.
Tool discipline	Correct use of inspect, interact, navigate, screenshot, and harvest tools.	Bad tool order can break playback even when reasoning looks correct.
Context handling	Ability to preserve page state, server attempts, iframe context, and failure reasons.	Long browser traces require stable state tracking.
Cache efficiency	Cache hit ratio and cached-token cost reduction.	Repeated prompts and tool descriptions should be reused instead of billed as new input every run.
Rate limits	Requests per minute, tokens per minute, and requests per day.	A model that works once may still be unusable for large batch evaluation.

Table 6.8: Model selection criteria for the regression batch.

long browser traces, use tools reliably, respond quickly, stay cheap during repeated website tests, and support caching well enough for large prompt/tool context.

Gemini 2.5 Flash-Lite was inexpensive but weak for this workflow: it often stopped early, missed page state, or failed to reason through chained navigation. Gemini 2.5 Flash was stronger and reasoned better, but it was more expensive for repeated batch evaluation. OpenRouter free models were useful for exploration, but most were not reliable enough for the main pipeline. GLM 4.5 Air, Devstral 2512, NVIDIA Nemotron 3 Super, GPT-OSS 120B, DeepSeek V3.2, and MiniMax M2.5 were tested, but the free endpoints were often slower, less consistent, or constrained by smaller effective context windows and routing limits. For this project, a model that is occasionally strong is less useful than a model that is fast, cheap, repeatable, and can handle long context every run.

Table 6.9: LLM model comparison.

Model tested	Ctx.	In / 1M	Cache in / 1M	Out / 1M	Evaluation note
Gemini 3.1 Flash-Lite	1,048k	$0.25	$0.025	$1.50	Final choice: 1M context, fast inference, cheap operation, good cache efficiency, and generous operating limits.
Gemini 2.5 Flash-Lite	1,048k	$0.10	$0.010	$0.40	Cheap, but performed poorly on multi-step navigation and tool-grounded reasoning.

Model tested	Ctx.	In / 1M	Cache in / 1M	Out / 1M	Evaluation note
Gemini 2.5 Flash	1,048k	$0.30	$0.030	$2.50	Good reasoning and stronger agent behavior, but higher cost for large repeated batches.
GLM 4.5 Air free	131k	$0.00	–	$0.00	Useful for trials, but constrained by smaller context and free-tier routing.
GLM 4.5 Air paid	131k	$0.13	$0.025	$0.85	Reasonable price, but not selected because context was smaller than Gemini.
Devstral 2512	262k	$0.40	$0.040	$2.00	Tested for coding-style reasoning, but not stable enough for the browser pipeline.
NVIDIA Nemotron 3 Super free	1,000k	$0.00	–	$0.00	Attractive context, but free route behavior was less predictable for repeated runs.
NVIDIA Nemotron 3 Super paid	1,000k	$0.09	–	$0.45	Cheap long-context option, but final pipeline used Gemini because of speed and integration behavior.
GPT-OSS 120B free	131k	$0.00	–	$0.00	Free, but small context for this workload and less reliable in browser-agent traces.
GPT-OSS 120B paid	131k	$0.039	–	$0.18	Very cheap, but context and behavior were not the best fit.
DeepSeek V3.2	131k	$0.252	$0.0252	$0.378	Tested, but not selected because long browser traces need larger context.
MiniMax M2.5 free	205k	$0.00	–	$0.00	Painfully slow inference and weak fit for fast browser loops.
MiniMax M2.5 paid	205k	$0.15	–	$1.15	Paid endpoint did not justify switching from Gemini for this workflow.

Prices and context windows in Table 6.9 are from OpenRouter's Models API, where pricing is reported per token and converted here to USD per million tokens [8]. The cache column uses OpenRouter's `input_cache_read` field when available; - means that no cache-read price was exposed for that model.

Google's Gemini pricing and caching references separate normal input, cached input, and output pricing, while the rate-limit reference describes RPM, TPM, and RPD as operational limits that vary by model and usage tier [10, 11, 9]. Those properties match the project constraints: the agents need a large context window for page state and traces, low latency for browser loops, low cost for repeated 200+ website regression batches, and enough rate-limit headroom for repeated experiments. That is why `google_genai::gemini-3.1-flash-lite` became the default model.

6.2.4 Cost Estimation

Cost is estimated from token usage and model pricing. Cached tokens are separated from new input tokens because provider-side caching changes the real price of repeated prompt and tool context.

$$
\begin{aligned}
Cost = \; & \frac{new\_input\_tokens}{1{,}000{,}000} \times input\_price \\
& + \frac{cached\_input\_tokens}{1{,}000{,}000} \times cached\_input\_price \\
& + \frac{cache\_write\_tokens}{1{,}000{,}000} \times cache\_write\_price \\
& + \frac{output\_tokens}{1{,}000{,}000} \times output\_price
\end{aligned}
$$

$$
CostPerWorkingWebsite = \frac{total\_estimated\_cost}{working\_websites}
$$

The dashboard exposes input cost, cached-input cost, cache-write cost, output cost, and total cost. In the dashboard snapshot, the model-spend view reports $4.505017 total recorded spend, $0.047926 average cost per run, 883 LLM calls, four provider names, and a blended cost of $0.000112 per 1k tokens.

Cost / token metric	Interpretation	Dashboard value
Total model spend	Sum of recorded or computed spend across model usage rows.	$4.505017
Average cost per run	Total spend divided by persisted runs with usage.	$0.047926
LLM calls	Number of model invocations recorded during the experiment period.	883
Average LLM calls per run	Approximate model invocations needed by a run.	8
Blended cost per 1k tokens	Total spend normalized by token volume.	$0.000112
Total displayed tokens	Dashboard token total for input and output usage.	40,220,868
New input tokens	Non-cached prompt/context tokens.	43,384,638
Cached input tokens	Input tokens served through prompt cache.	28,928,333
Output tokens	Generated model tokens.	454,914
Cache hit rate	Cached input divided by total input considered by the cache view.	40.0%
Average tokens per run	Token volume normalized by persisted run count.	386,739

Table 6.10: Observed cost, token, and cache metrics from the dashboard.

The important point is that cache is evaluated as part of model selection. A model that is accurate but repeatedly consumes full prompt and tool context can become too expensive for large batch testing. A useful configuration keeps the website success rate stable while raising the cache hit rate and lowering cost per working website.

6.2.5 Provider Intelligence Test

Provider analysis is evaluated after stream or embedded URLs are collected. The test takes the extracted URLs, resolves their hostnames, checks whether a routable IP is found, and then enriches the result with provider and country information using IP and registration data sources such as IPinfo and RDAP/WHOIS lookups [30, 31]. This matters because a stream URL is stronger evidence when it can be connected to a hosting provider, CDN, or network operator.

Provider metric	Meaning	Dashboard value
Checked URLs	Stream or media URLs inspected by the provider lookup flow.	694
Resolved IPs	URLs that resolved to a routable IP address.	661
Provider matches	URLs mapped to an organization or hosting provider.	637
Abuse contacts	Abuse email contacts found through the lookup path.	0
Provider coverage	Provider matches divided by checked URLs.	637 / 694 = 91.8%
IP resolution coverage	Resolved IPs divided by checked URLs.	661 / 694 = 95.2%

Table 6.11: Provider-intelligence telemetry used to validate stream URL enrichment.

The dashboard also shows the distribution of infrastructure behind the extracted URLs. The largest providers in the observed snapshot were DDOS-GUARD LTD, StormWall s.r.o., Fastly, Cloudflare, SpectraIP B.V., Akamai, Private Layer INC, and Amazon. Country distribution was also recorded, with the largest groups in Russia, the United States, and the Netherlands. These values do not decide whether a website works, but they validate the post-extraction enrichment layer.

6.3 Tool Evaluation

Tool evaluation also uses the website batch. The question is whether a tool extracts what it was designed to extract on real website layouts. A tool is useful only if it helps the agent satisfy the website acceptance rule: classify, navigate, activate playback, return network evidence, and capture a screenshot-supported result.

6.3.1 Tool Success Criteria

The tool dashboard is used to verify that this layer is not silently failing while the agent is reasoning. In the observed dashboard, the browser/tool layer recorded 1,410 tool calls with a 96.7% success rate. There were 46 failed calls, eight distinct tools, an average duration of 6.51 seconds per call, and 14 tool calls per pipeline run.

Tool success rate alone is not enough, because a tool can technically succeed while extracting the wrong evidence. For example, a successful screenshot call does not mean the player was activated. Therefore tool evaluation combines dashboard health with website acceptance: the tool must return the expected kind of evidence and the website must still pass the batch rule.

Tool family	Expected extraction	Batch success signal
Inspect tools	Compact DOM structure, visible buttons, links, match cards, iframes, player controls, and page hints.	Agent receives enough context to choose the next correct action without overloading the model.
Navigation tools	Open target URLs, follow host candidates, keep same-page tab state when needed.	Agent reaches the correct landing, hosting, or embedded page.
Interaction tools	Click play, close popups, switch servers, activate tabs, restore focus after ad windows.	Player state changes or a meaningful blocker is documented.
Screenshot tools	Capture visual state at classification, player activation, and final evidence moments.	Screenshot supports why the website passed or failed.
Harvest tools	Collect network entries, media URLs, iframe URLs, manifests, and performance evidence.	At least one useful network, embedded, or media candidate is returned when playback is available.
Provider tools	Enrich host and stream evidence with IPInfo, RDAP, WHOIS, or provider metadata.	Evidence can be linked to provider or abuse-contact context when available.

Table 6.12: Tool-family evaluation criteria for batch-level testing.

Tool metric	What it means	Dashboard value
Total calls	Total observed browser/tool invocations.	1,410
Successful calls	Tool calls that returned a successful outcome.	1,364
Failed calls	Tool calls that returned an error or failed outcome.	46
Tool success rate	Successful calls divided by total calls.	96.7%
Unique tools	Distinct tool names observed in the dashboard.	8
Average duration	Mean observed duration per tool call.	6.51 s
Calls per run	Average tool calls per pipeline run.	14

Table 6.13: Observed tool-health telemetry from the dashboard.

6.3.2 Tool Regression Findings

The main tool failures were behavioral. The tools often worked technically, but the agent used them at the wrong time or for the wrong page type. Regression testing catches these cases because a website that previously returned evidence stops passing the batch.

Observed issue	Regression caused	Fix applied
`harvest()` before play click	Website fails because no runtime stream has been generated yet.	Added activation guard and mandatory media-state check.
Generic inspect on hosting pages	Server controls, tabs, and player-specific elements are missed.	Added page-profiled inspect behavior and clearer tool descriptions.
Repeated screenshots as waiting	Higher cost without new evidence and no playback progress.	Added explicit wait/checkpoint logic and reduced screenshot loops.
`navigate()` for same-page tabs	Page reloads or loses state instead of opening dynamic server content.	Clarified `interact` versus `navigate`.
Ignoring popup or new-tab focus	Agent reasons over the advertisement page instead of the original player page.	Added popup close, focus restore, and JavaScript-click fallback.

Table 6.14: Tool-level regressions found through website-batch testing.

Tool change	Before	After	Meaning
Activation guard on `harvest`	65% misuse	12% misuse	Fewer websites failed because harvesting happened before playback.
Specialized hosting inspection	40% missed controls	8% missed controls	Server buttons and player controls were found more reliably.
Clarified `navigate` versus `interact`	30% misuse	5% misuse	Same-page tabs and dynamic controls were handled without unnecessary reloads.

Table 6.15: Tool-description improvements measured against the website batch.

6.4 Chapter Conclusion

The evaluation shows that the system must be judged by website-level evidence collection, not by isolated tool counts or a single classifier score. A website works only when the agents classify it, navigate through its layout, activate or inspect the player, recover from popups or redirects, extract media/network evidence, and return screenshot-supported results.

The 200+ website batch is the central evaluation mechanism. It supports regression testing because every prompt, model, and tool change is checked against old and new websites. This matches the real engineering goal: add support for more layouts without losing sites that already worked.

Model evaluation adds another layer by comparing accuracy, consistency, cache hit ratio, rate limits, and cost per working website. Tool evaluation checks whether each browser tool extracts the evidence it is responsible for on real pages. Together, these three evaluation points provide a practical way to improve the system without relying on one-off successful demos.

CHAPTER 7

General Conclusion

This PFE internship addressed a practical evidence-collection problem in the sports rights-protection domain. Soft Stars already had crawler-based monitoring capabilities, but the project showed why a crawler alone is not enough for many illegal streaming websites. The most useful evidence is often not present in the first HTML response. It appears after a rendered page loads scripts, after a channel group is opened, after a server button is selected, after a player is activated, or after an embedded iframe requests a short-lived media manifest. The central problem was therefore not only to find suspicious URLs, but to preserve a reviewable browser trace that explains how the evidence was reached.

The work followed Design Science Research Methodology as the sole methodological backbone. The organizational problem was first identified through the limitations of static crawling and manual review. The solution objectives were then defined around classification, browser interaction, screenshots, stream evidence, channel metadata, provider enrichment, traceability, and review. The artifact was designed and developed in two steps. The n8n workflow validated the idea quickly and gave the company a landing-agent backup path beside the crawler. Open Web Catcher (OWC) then organized the same work into a complete platform made of FastAPI, Next.js, PostgreSQL, MCP browser tools, Puppeteer/Chrome runtime, specialist agents, model telemetry, and evidence outputs.

The main contribution is the separation of the investigation into clear page roles and agent responsibilities. The orchestrator controls the run, prepares handoffs, manages queues, and aggregates results. The classification agent decides whether a URL is a landing, hosting, embedded, or irrelevant page. The landing agent discovers match, event, channel, hosting, or embedded candidates. The hosting agent activates server and player controls while preserving screenshots and diagnostics. The embedded agent stays inside direct player contexts and harvests final media evidence. Provider enrichment and takedown-email drafting are downstream review stages, not automatic enforcement actions.

OWC also contributes a more reliable browser-tool layer. The tool profiles separate broad inspection, precise context reads, navigation, interaction, screenshot capture, media-state checks, stream harvesting, and memory hints. Browser runtime controls handle profile-scoped sessions, fingerprint behavior, proxy validation, sticky proxy selection, popup recovery, iframe recovery, and media verification. This made the workflow more explainable: a failure could be linked to classification, a missing hosting candidate, a player activation problem, an iframe issue, a proxy failure, a missing stream, or a skipped provider stage instead of being reduced to a vague failed scrape.

The implementation also produced a review-oriented operator experience. The console shows run status, active traces, screenshots, streams, model usage, cached tokens, costs, tool calls, provider results, persistence health, and runtime events. This is important because evidence

collection is not useful if the operator cannot understand what happened after the browser closes. The database is therefore used for observability and review: it stores runs, agent runs, runtime events, LLM calls, tool calls, screenshots, streams, provider analyses, email drafts, prompt compilations, and memory hints.

The company deployment decision remained pragmatic. The complete OWC platform defines the long-term architecture, but the operational deployment focused on the validated landing agent. Target URLs arrive through the `agent_tasks` Azure Service Bus queue, the container runs n8n, the landing-agent worker, Puppeteer MCP, and Chrome, and the extracted matched URLs with metadata are returned through `agent_results`. This keeps the existing crawler, the n8n backup workflow, and the AI browser agent decoupled while still giving the company a path for dynamic pages that require rendered-state reasoning.

The project remains a foundation rather than a finished production enforcement platform. Some pages will still fail because of anti-bot behavior, unstable player code, token expiry, misleading channel labels, popup-heavy navigation, or unavailable providers. Human review is still necessary before evidence is used operationally. The evaluation also needs to grow over larger controlled batches before strong accuracy or cost claims can be made. However, the internship achieved its main objective: it turned a fragile investigation problem into a structured artifact with page-role separation, browser evidence, traces, screenshots, stream extraction, channel detection, provider enrichment, deployment direction, and clear limitations.

The future work is therefore concrete. OWC should be evaluated on larger labeled website batches, with stronger regression checks for classification, landing discovery, hosting activation, embedded extraction, channel detection, stream yield, provider coverage, tool success, token usage, and cost per run. Browser memory can be improved so successful selectors and server patterns are reused without hardcoding sites. The operator console can add screenshot annotation for reviewers. Deployment can expand from the landing-agent backup toward more queue-driven specialist agents once the company decides that the operational risk and cost are justified.

Overall, the internship demonstrated that an AI-assisted browser workflow is a suitable complement to crawler-based monitoring when it is constrained, observable, and designed around evidence rather than around unconstrained automation. The value of OWC is not that it permanently solves every illegal streaming website. Its value is that it gives Soft Stars a clear engineering foundation for investigating dynamic streaming pages: classify the page, follow controlled handoffs, interact with the browser, verify the player state, extract evidence, preserve the trace, and make the result understandable for human review.

References

Bibliography

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023. Available: https://arxiv.org/abs/2210.03629

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Ciaramita, L. Zettlemoyer, H. Schütze, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. Available: https://arxiv.org/abs/2302.04761

[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024. Available: https://arxiv.org/abs/2308.11432

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. Available: https://arxiv.org/abs/2201.11903

[5] Anthropic, *Claude 3 Model Card*, Anthropic, 2024. Available: https://www.anthropic.com/claude

[6] Google DeepMind, *Gemini: A Family of Highly Capable Multimodal Models*, Technical Report, Google DeepMind, 2023. Available: https://arxiv.org/abs/2312.11805

[7] Google, *Gemini 3.1 Flash-Lite: Built for Intelligence at Scale*, The Keyword, 2026. Available: https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-flash-lite/

[8] OpenRouter, *Models API*, Official documentation, 2026. Available: https://openrouter.ai/api/v1/models

[9] Google, *Gemini API Rate Limits*, Google AI for Developers documentation, 2026. Available: https://ai.google.dev/gemini-api/docs/rate-limits

[10] Google, *Gemini Developer API Pricing*, Google AI for Developers documentation, 2026. Available: https://ai.google.dev/gemini-api/docs/pricing

[11] Google, *Context Caching in the Gemini API*, Google AI for Developers documentation, 2026. Available: https://ai.google.dev/gemini-api/docs/caching

[12] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: https://pptr.dev

[13] Microsoft, *Playwright Documentation*, Official documentation, 2026. Available: https://playwright.dev/docs/intro

[14] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A Realistic Web Environment for Building Autonomous Agents," in *Proceedings of ICLR*, 2024. Available: https://arxiv.org/abs/2307.13854

[15] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2Web: Towards a Generalist Agent for the Web," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[16] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, "WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[17] T. Le Sellier de Chezelles, M. Gasse, A. Drouin, and others, "The BrowserGym Ecosystem for Web Agent Research," *arXiv preprint arXiv:2412.05467*, 2024.

[18] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: https://chromedevtools.github.io/devtools-protocol/

[19] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: https://www.jstor.org/stable/25148625

[20] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: https://doi.org/10.2753/MIS0742-1222240302

[21] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: https://fastapi.tiangolo.com

[22] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: https://nextjs.org/docs

[23] LangChain, *LangChain Agents and Tools Documentation*, Official documentation, 2026. Available: https://docs.langchain.com/oss/python/langchain/agents

[24] LangChain, *LangGraph Overview*, Official documentation, 2026. Available: https://docs.langchain.com/oss/python/langgraph/overview

[25] Model Context Protocol, *Model Context Protocol Specification*, Official documentation, 2026. Available: https://modelcontextprotocol.io/specification/2024-11-05/basic/index

[26] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: https://docs.n8n.io

[27] Microsoft, "Jobs in Azure Container Apps," *Microsoft Learn*, 2024. Available: https://learn.microsoft.com/en-us/azure/container-apps/jobs

[28] Microsoft, "Azure Service Bus Messaging Service," *Microsoft Learn*, 2024. Available: https://learn.microsoft.com/en-us/azure/service-bus-messaging/

[29] Cloudinary, *Cloudinary Upload API Documentation*, Official documentation, 2026. Available: https://cloudinary.com/documentation

[30] IPinfo, *IPinfo API Documentation*, Official documentation, 2026. Available: https://ipinfo.io/developers/ipinfo-api

[31] ICANN, *Registration Data Access Protocol (RDAP)*, Official documentation, 2026. Available: https://www.icann.org/rdap/

[32] Cloudflare, *How Cloudflare Challenges Work*, Official documentation, 2026. Available: https://developers.cloudflare.com/cloudflare-challenges/concepts/how-challenges-work/

[33] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: https://www.rfc-editor.org/rfc/rfc8216

[34] ISO/IEC, *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, ISO/IEC 23009-1, 2022.

[35] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: https://github.com/gorhill/uBlock

[36] MUSO, *Global Piracy Insights: Sports Streaming Report*, MUSO Intelligence, 2023. Available: https://www.muso.com/insights

[37] European Audiovisual Observatory, *Trends in the Field of Audiovisual Piracy in Europe*, Council of Europe, Strasbourg, 2022. Available: https://rm.coe.int/

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn – E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax : +216 70 685 454